WAINLUX K6 Web abstraction on Pi Zero W

Table of Contents

1. Summary . . . 4
   1.1. Clean-Room Statement . . . 5
2. WARNINGS . . . 5
3. First Card: Preview, Mark, Then Burn . . . 6
4. Architecture . . . 7
5. Clone, Build, Deploy . . . 9
6. Getting Started . . . 10
   6.1. For Users . . . 10
      6.1.1. Deploy to Pi . . . 10
   6.2. For Developers . . . 10
      6.2.1. Read Code . . . 10
      6.2.2. Generate Diagrams . . . 10
   6.3. For Documentation Writers . . . 10
      6.3.1. Add Diagram . . . 10
   6.4. For Contributors . . . 11
      6.4.1. Clone Repository . . . 11
      6.4.2. Follow Style . . . 11
   6.5. Quick Commands . . . 11
7. Technical Reference and Evidence . . . 11
   7.1. Discover . . . 12
   7.2. Submit and Review . . . 12
   7.3. Types . . . 13
   7.4. Execute One Stage . . . 13
   7.5. Trust and Lifetime . . . 14
   7.6. Tests . . . 14
8. K6 Raster Protocol . . . 14
   8.1. Link and Coordinates . . . 14
   8.2. Response Bytes . . . 15
   8.3. Commands Used by the Driver . . . 15
   8.4. JOB_HEADER (0x23) . . . 16
   8.5. DATA (0x22) . . . 17
   8.6. Framing, Stop and Status Diagrams . . . 18
   8.7. Evidence Limits . . . 19
9. Hardware Observations . . . 19

9.1. Current K6 Profile . . . 20
9.2. Executed Rust Ladder Tests . . . 20
9.3. 9b: Height and Gap Comparison . . . 23
9.4. Card Result . . . 24
9.5. Historical Cork and PLA Close-ups . . . 25
9.6. Historical QR Run . . . 27
10. Burn Timing . . . 28
10.1. Current Report . . . 28
10.2. Historical Measurements . . . 28
Appendix A: Architecture Decisions . . . 29
11. ADR-002: K6 Serial Transport . . . 29
11.1. Status . . . 29
11.2. Decision . . . 29
11.3. Consequences . . . 29
12. ADR-006: Clean-Room Protocol Work . . . 29
12.1. Status . . . 29
12.2. Decision . . . 29
12.3. Consequences . . . 30
13. ADR-007: AsciiDoc . . . 30
13.1. Status . . . 30
13.2. Decision . . . 30
13.3. Consequences . . . 30
14. ADR-010: Bytefield Protocol Diagrams . . . 30
14.1. Status . . . 30
14.2. Decision . . . 30
14.3. Consequences . . . 30
15. ADR-013: Simulator Testing . . . 31
15.1. Status . . . 31
15.2. Decision . . . 31
15.3. Consequences . . . 31
16. ADR-019: HTTP Review Boundary . . . 31
16.1. Status . . . 31
16.2. Decision . . . 31
16.3. Consequences . . . 31
17. ADR-021: Rust Service . . . 32
17.1. Status . . . 32
17.2. Decision . . . 32
17.3. State and lifetime . . . 32
17.4. Consequences . . . 32
18. ADR-022: ARM Packaging . . . 32
18.1. Status . . . 32

18.2. Decision . . . 33
18.3. Consequences . . . 33
19. ADR-025: Card Placement . . . 33
19.1. Status . . . 33
19.2. Decision . . . 33
19.3. Consequences . . . 33
20. ADR-026: Network MCP Hub . . . 34
20.1. Status . . . 34
20.2. Decision . . . 34
20.3. Consequences . . . 34
Appendix B: Requirements . . . 34
B.1. Functional . . . 34
B.2. Non-Functional . . . 34
21. FR-001: Reviewed Raster Execution . . . 35
21.1. Status . . . 35
21.2. Requirement . . . 35
21.3. Acceptance . . . 35
21.4. Scope . . . 35
22. FR-002: WiFi QR Composition . . . 35
22.1. Status . . . 35
22.2. Requirement . . . 35
22.3. Acceptance . . . 35
22.4. Evidence . . . 36
23. FR-003: Progress and Timing . . . 36
23.1. Status . . . 36
23.2. Requirement . . . 36
23.3. Acceptance . . . 36
24. FR-007: Stop and Cancellation . . . 36
24.1. Status . . . 36
24.2. Requirement . . . 36
24.3. Acceptance . . . 37
25. FR-009: Test Ladder . . . 37
25.1. Status . . . 37
25.2. Requirement . . . 37
25.3. Acceptance . . . 37
26. FR-011: Material Placement . . . 37
26.1. Status . . . 37
26.2. Requirement . . . 38
26.3. Acceptance . . . 38
27. FR-013: Hardware-Independent Testing . . . 38
27.1. Status . . . 38

27.2. Requirement . . . 38
27.3. Acceptance . . . 38
28. FR-015: Operator Test Verdicts . . . 38
28.1. Status . . . 38
28.2. Requirement . . . 39
28.3. Acceptance . . . 39
29. FR-016: Reviewed Card Jobs . . . 39
29.1. Status . . . 39
29.2. Requirement . . . 39
29.3. Acceptance . . . 39
29.4. Trust Limit . . . 40
30. NFR-002: Hardware-Free Test Isolation . . . 40
30.1. Status . . . 40
30.2. Requirement . . . 41
30.3. Acceptance . . . 41
31. NFR-005: Redacted Run History . . . 41
31.1. Status . . . 41
31.2. Requirement . . . 41
31.3. Acceptance . . . 41
Appendix C: Images . . . 41
Appendix D: Glossary . . . 42
Appendix E: Copyright and License . . . 43
E.1. Code License . . . 43
E.2. Protocol Documentation License . . . 44
E.3. Clean Room Statement . . . 44
E.4. Copyright Notice . . . 44
Appendix F: MIT License . . . 44
Appendix G: Protocol Documentation License . . . 45
Appendix H: Bytefield-SVG . . . 45
32. Bytefield-SVG Container . . . 45
32.1. Build . . . 45
32.2. Usage . . . 46
32.3. Helper Script . . . 46
32.4. Flatpak Users . . . 46
32.5. Examples . . . 46

Other formats: HTML | GitHub Repository

1. Summary

Open-source host software for operating Wainlux K-series laser engravers using an independently

developed protocol implementation.



This project is an independently developed, clean-room implementation created through analysis of publicly observable behavior and legally obtained software. It is not affiliated with or endorsed by Wainlux.

Is:

- Host-side control software
- Protocol implementation
- Interoperability tool

Is not*:*

- ~~Firmware replacement~~
- ~~Vendor software redistribution~~
- ~~Reverse-engineered source release~~

The current service is a standalone Rust application for the Pi Zero W. It serves the operator UI and HTTP API on port 8081, composes and reviews card jobs, and controls the K6 over USB serial. See [api-contract].

The laser service runs from `docker/docker-wainlux-rs`. Continue to Clone, Build, Deploy for deployment, then Technical Reference and Evidence for the protocol and test record. A separate MFA backup tool can provide recovery-code content.

1.1. Clean-Room Statement

Project owner and decision authority: Sean Donnellan. Sean triggered and verified the physical K6 tests. Protocol behavior was established from packet captures, serial monitoring, black-box tests and public comparative material. GitHub Copilot performed isolated Ghidra analysis and separately assisted with implementation; only protocol descriptions were handed between those roles. No vendor source, binary, Ghidra project or analysis artifacts are distributed. This was tool-mediated role separation, not a formal two-human clean-room program. See the included ADR-006: Clean-Room Protocol Work decision for the implementation boundary.

2. WARNINGS



- Laser safety: **always** wear appropriate laser safety glasses.
- Fire hazard: do not run unattended; have a fire-safe surface.
  - Keep a fire extinguisher handy, or a fire blanket.
- Use a smoke detector.

3. First Card: Preview, Mark, Then Burn

The operator reviews each exact raster and starts each stage separately. For a first card, burn the optional guide on bare backing, confirm the machine has stopped, place the card against the guide, then review and start the content stage.

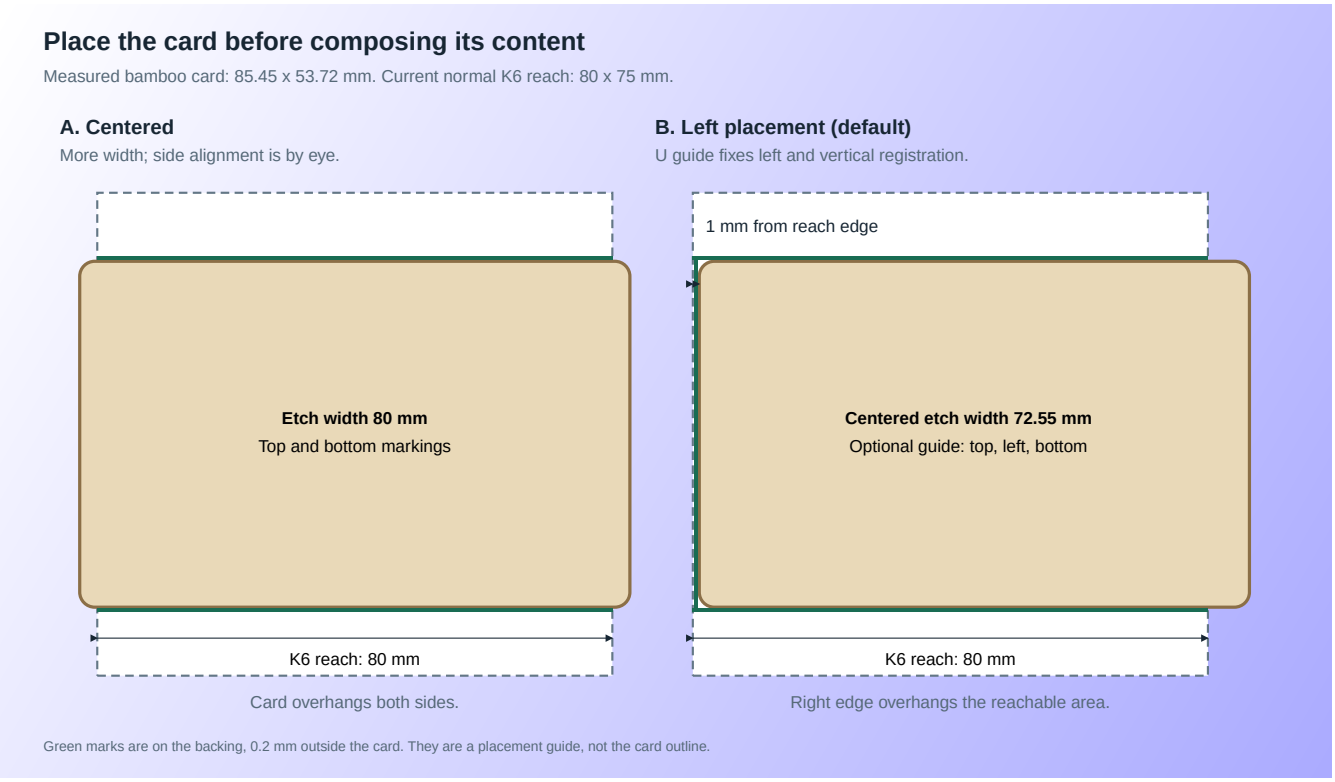


Figure 1. Choose a card placement before composing its content

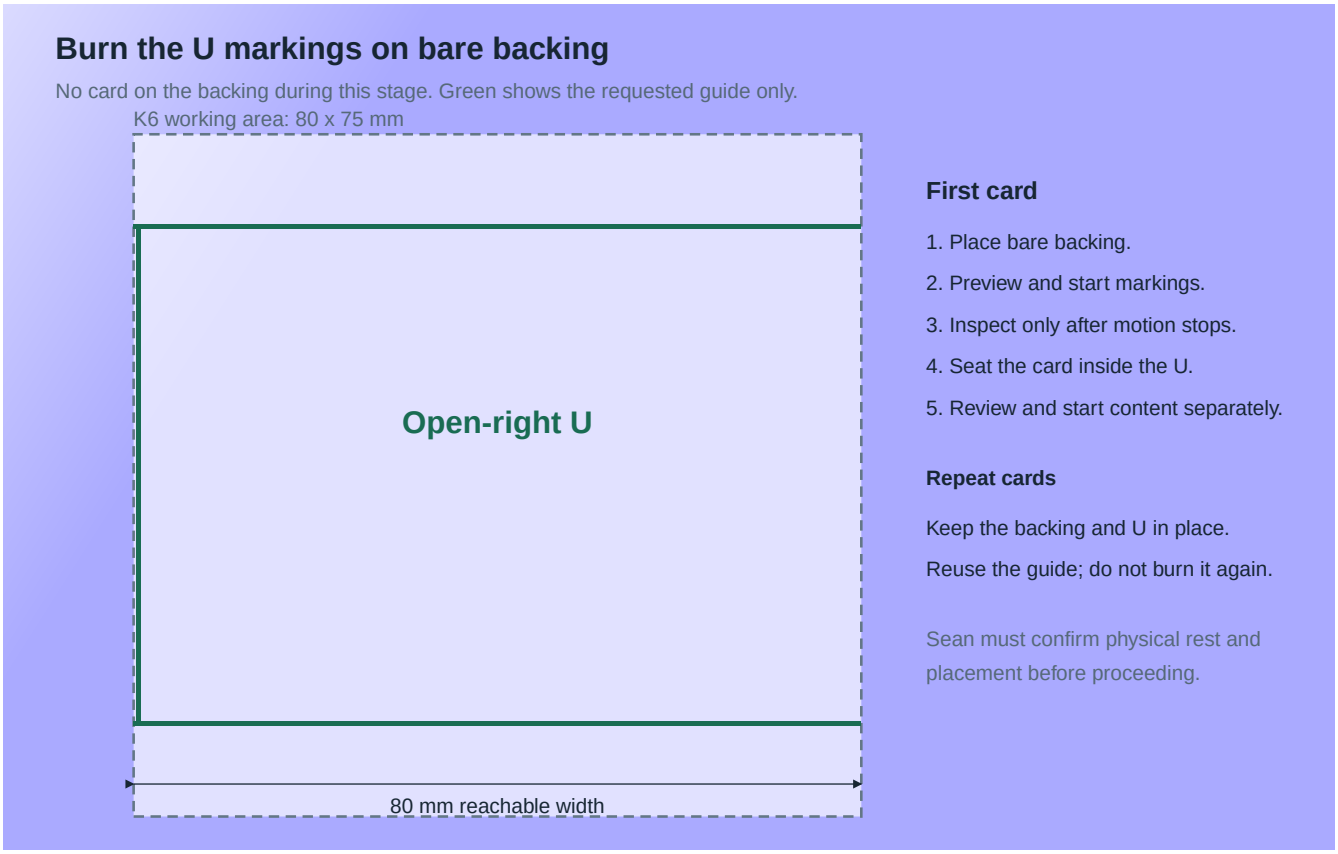


Figure 2. The first stage marks the bare backing; the card is not present

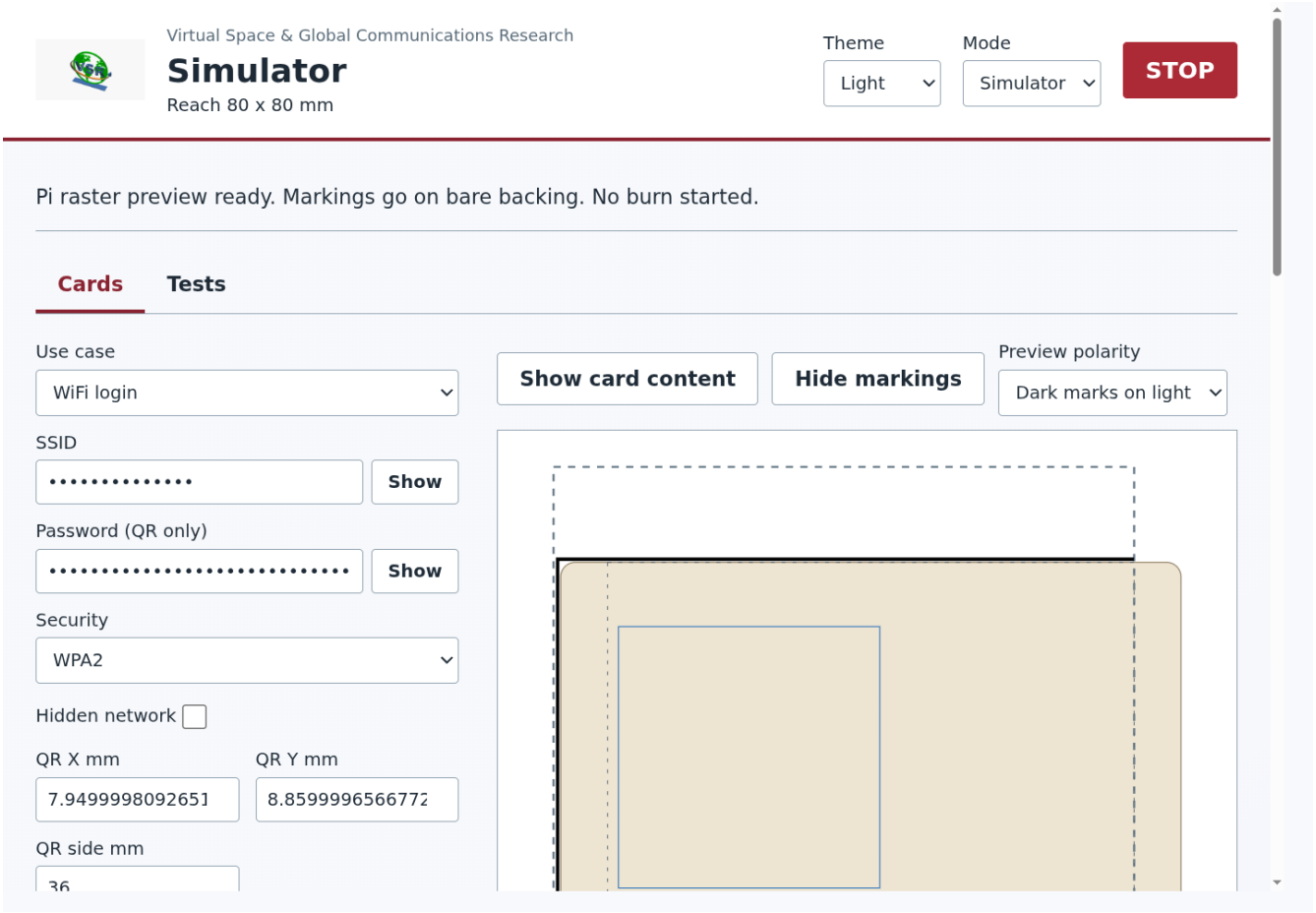


Figure 3. Rust operator UI with a synthetic Wi-Fi preview; simulator mode, no burn started

The UI and API are served at `http://<pi-ip>:8081`. The complete Pi service contract is included at [api-contract].

4. Architecture

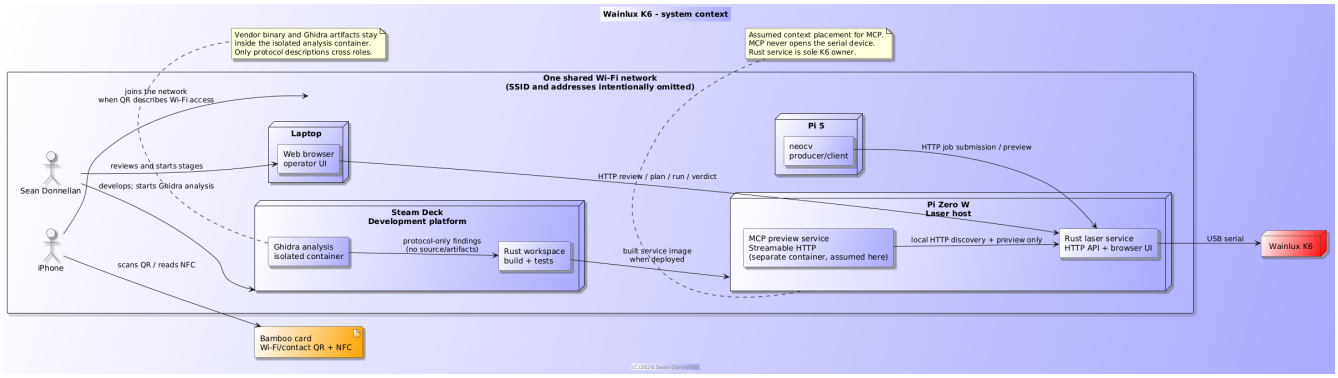


Figure 4. System context

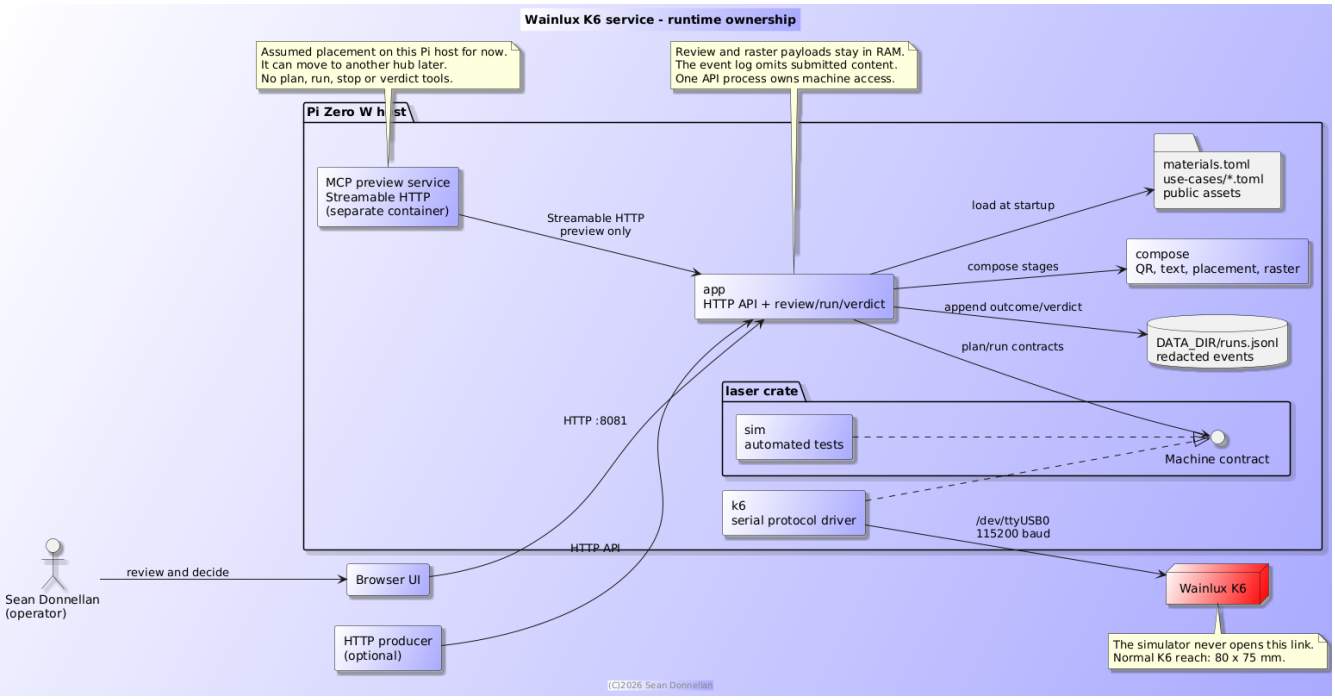


Figure 5. Rust service and device ownership

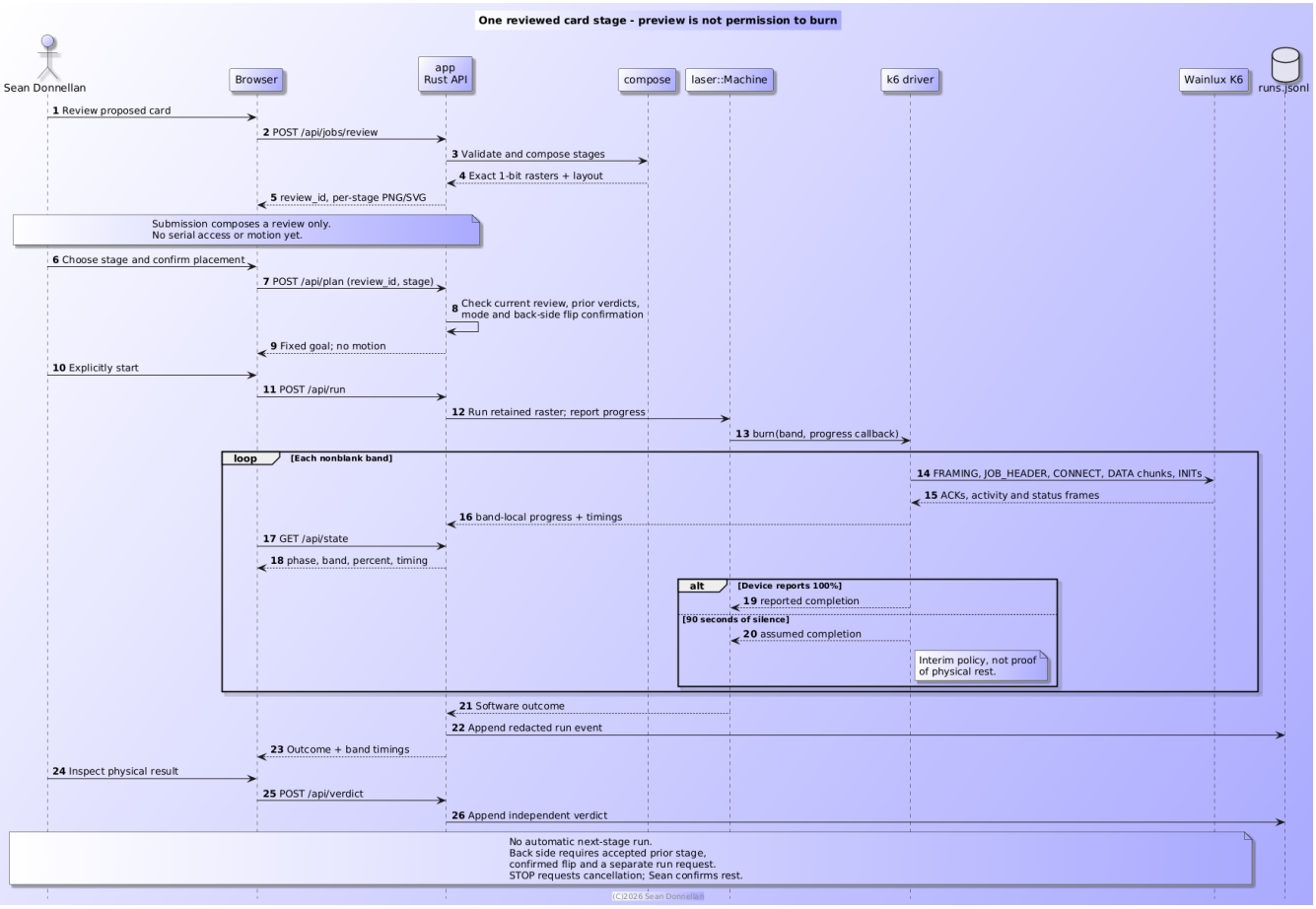


Figure 6. One reviewed stage from API submission through physical verdict

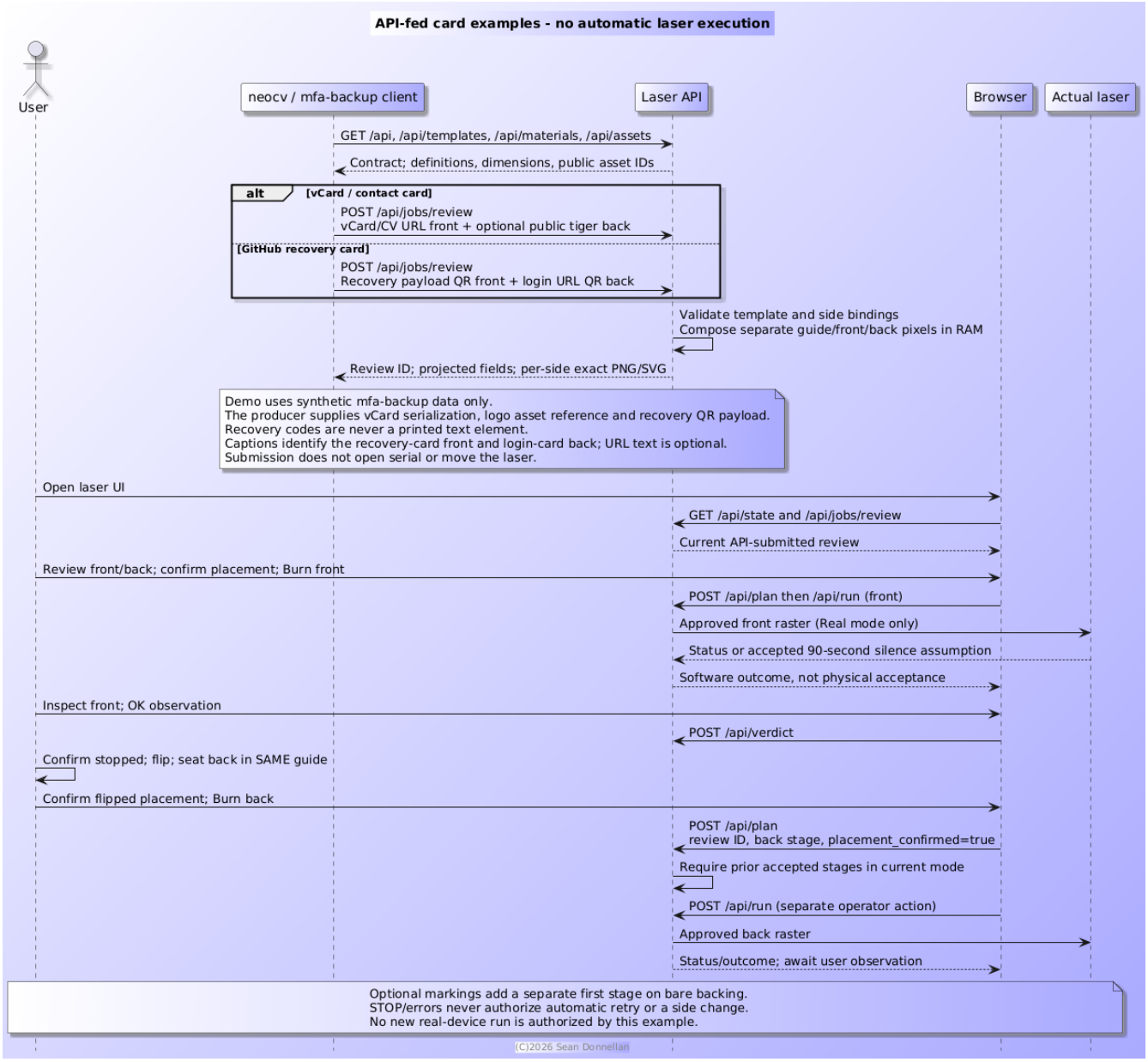


Figure 7. Operator UI and external clients share the API

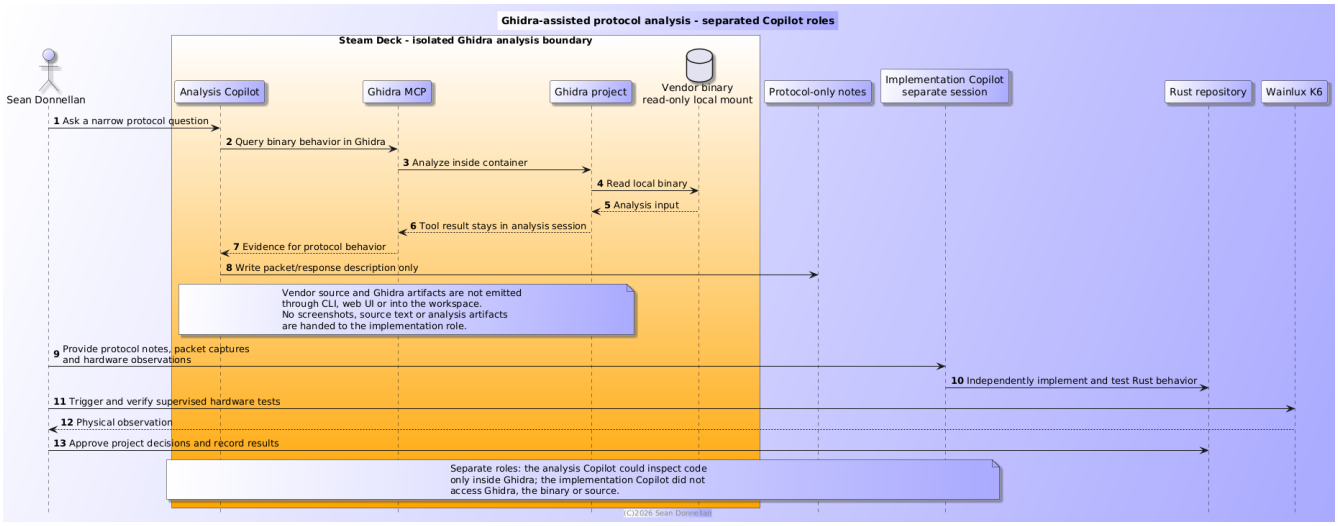


Figure 8. Clean-room analysis and implementation handoff

5. Clone, Build, Deploy

6. Getting Started

Quick guide to using this repository.

6.1. For Users

6.1.1. Deploy to Pi

Start the preloaded ARM image

```
cd docker/docker-wainlux-rs
docker compose up -d
```

Access: `http://<pi-ip>:8081`. The ARM image must already be loaded on the Pi; this compose file does not build it.

6.2. For Developers

6.2.1. Read Code

- Service API and operator workflow
- K6 Raster Protocol for K6 serial behavior
- First Card: Preview, Mark, Then Burn for the embedded operator UI preview

Run `cargo test --workspace` from `docker/docker-wainlux-rs`.

6.2.2. Generate Diagrams

Render the documentation diagrams

```
plantuml documentation/images/*.plantuml
```

Output: `documentation/images/*.png`

6.3. For Documentation Writers

6.3.1. Add Diagram

Create `.plantuml` file in `documentation/images/`:

Minimal source for a PlantUML diagram

```
@startuml

actor User
```

```
User --> System
@enduml
```

Render a new PlantUML diagram

```
plantuml images/new-diagram.plantuml
```

Reference the rendered diagram from AsciiDoc

```
\image::documentation/images/new-diagram.png[Description]
```

6.4. For Contributors

6.4.1. Clone Repository

Clone the repository

```
git clone <repo-url> wainlux-pi
cd wainlux-pi
```

6.4.2. Follow Style

- DRY - Don't Repeat Yourself
- KISS - Keep It Simple, Stupid
- Hemingway - Short, direct sentences
- No fluff - Essential info only

6.5. Quick Commands

Run documentation and Rust checks

```
# Check documentation
./scripts/test_docs.sh

# Test Rust workspace
cd docker/docker-wainlux-rs && cargo test --workspace
```

7. Technical Reference and Evidence

Progress percentages are scoped to the current band, not the full side/card. `GET /api/state` exposes terminal band records in `band_timings` and the run outcome. Each record includes `transfer_ms` (raster transfer), `setup_ms` (preparation and protocol setup outside transfer and wait), `wait_ms` (completion-status wait), and `elapsed_ms` (total band time). Sum the fields across bands for a run

total. All values are milliseconds.

After 90 seconds of silence, completion is reported as assumed, not device-confirmed. The UI labels this separately. If a band was assumed complete, the following band event indicates that fact. = Laser Service API

Standalone Pi Zero service. HTTP clients submit content; the operator reviews exact 1-bit rasters and starts each burn separately. `GET /api` serves this contract. The browser and external clients use the same endpoints. `GET /openapi.yaml` serves the OpenAPI 3.1 contract. `GET /docs` serves bundled Swagger UI 5.34.1; Try It Out is enabled for GET only. License notices are in `app/assets/swagger-ui/`.

7.1. Discover

Request	Response
`GET /api/machine`	Reach, resolution, settings and capabilities.
`GET /api/materials`	Material/card definitions and resolved machine settings.
`GET /openapi.yaml`	OpenAPI 3.1 schema for Swagger-compatible clients; no UI bundle is included.
`GET /api/templates`	Complete field, element, side and edit/reveal policy definitions.
`GET /api/assets`	Reusable public assets. Initially the prepared 1-bit tiger.
`GET /api/assets/tiger.png`	Public tiger pixels, prepared from the repository's existing example.

7.2. Submit and Review

Two submission forms are available. Neither opens the serial connection or starts the laser.

- Producer: `POST /api/jobs/review` with a `job` object: `template`, `piece`, `backing`, `placement`, `guide`, `items`, optional `values`.
- Template form: `POST /api/jobs/form` with `template`, `review_id` (`null` for a new idle review), `piece`, `backing`, `placement`, `guide`, and changed `fields` keyed by template field ID. Field edit policy is enforced.

Each item names a declared element `id`, `side` (`front` or `back`) and `content`. Content kinds are `qr` (`text` payload), `text` (`text`, `font_size_mm`, optional `font`), `wifi` (network fields) or `image` (`asset_id`). Positions and dimensions omitted by a producer come from the template. An image asset ID refers to prepared pixels; it is not an arbitrary URL or Pi path.

Example vCard job with a prepared logo asset

```
{
  "job": {
    "template": "vcard",
    "piece": "bamboo-ntag215-light",
    "backing": "backing-first",
    "placement": "left",
```

```
    "guide": false,
    "items": [
      {"id":"contact-qr","side":"front","content":{"kind":"qr","text":"BEGIN:VCARD
\nVERSION:3.0\nFN:Jane Example\nURL:https://example.com/cv\nEND:VCARD"}},
      {"id":"logo-back","side":"back","content":{"kind":"image","asset_id":"tiger"}}
    ]
  }
}
```

The response contains `review_id`, `template`, policy-projected `fields`, geometry-only `job`, `layout`, `display`, ordered `stages` and `accepted_stages`. Each stage has `name`, `side`, `goal`, resolved `values`, and `burns` with PNG/SVG pixels, pixel dimensions and physical position/size. Different sides are separate rasters, never composited together. `guide=true` adds an optional guide stage before the front; existing markings can be reused with `guide=false`.

`GET /api/jobs/review` retrieves the current review. The UI discovers it from `GET /api/state` and loads it without a browser-file injection. `POST /api/jobs/form` edits and regenerates a review using its current ID. Stale IDs, undeclared fields and forbidden edits are rejected. `DELETE /api/jobs/review` explicitly discards an idle review; an active attempt blocks replacement/discard.

7.3. Types

- `wifi-login`: network QR plus independently optional readable labels. Password is QR-only.
- `vcard`: producer-supplied vCard text or CV URL, optional readable contact label, optional logo back. The producer owns vCard serialization/escaping and contact data; the laser does not query neocv's graph.
- `github-recovery`: recovery payload QR on the front and login URL QR on the back. Small required-by-default captions identify "GitHub MFA recovery codes" and "GitHub login to use these codes". Recovery codes are transport data, not printable page text. A second small, optional caption can print the login URL. An optional front checklist adds 16 blank boxes (2 columns x 8 rows) beside the recovery QR for manually marking used codes. It defaults off and does not contain or encode code values. A producer opts in by submitting a `use-checklist` item with `kind: checkboxes`, `count: 16`, `columns: 2`, `box_mm: 3.5`, `gap_mm: 0.8`, and `line_mm: 0.35` at the declared `check-x` / `check-y` position.

For `mfa-backup`, use only `examples/synthetic-codes.txt` for demonstrations. It is not a valid GitHub recovery-code set. The whole text is submitted as `recovery-payload` for the front QR. In the operator form it is masked by default, but editable and explicitly revealable so a user entering real codes by hand can check their input. The QR preview still visibly carries those codes. The client submits both `recovery-qr` (front) and `login-qr` (back) items. Small front/back purpose captions are on by default and carry no secret data; `include-url-text` is off by default and can add the URL as an additional caption. Real recovery formats and line lengths must be previewed; overflow is rejected, not silently shrunk. Private payloads remain in RAM, not template files or operational logs.

7.4. Execute One Stage

1. Review both sides before starting. Confirm placement.

2. `POST /api/plan` with `review_id`, `stage` (zero-based) and `placement_confirmed` (`true` required for a back stage).
3. Only `POST /api/run` starts the planned stage. It never advances automatically to another user stage.
4. Poll `GET /api/state`. `POST /api/stop` cancels queued driver work and requests stop; the operator confirms physical rest.
5. Record `POST /api/verdict` with `status` (`ok`, `deviation`, or `skipped` for an unperformed plan) and an observation.
6. A later stage requires accepted prior stages. Inspect the front, confirm motion stopped, flip the card and seat it against the same guide before planning the back.

Plan the reviewed front stage

```
{"review_id":"returned-review-id","stage":1,"placement_confirmed":true}
```

An edited review clears stage acceptance. Re-running an earlier stage invalidates later acceptance. Simulator/Real mode changes clear acceptance too. The K6's temporary completion policy assumes completion after 90 seconds without a response; it does not prove a physical OK.

7.5. Trust and Lifetime

This is a trusted-LAN service, not a public authenticated API. Producer/operator role separation and session leases remain open. Non-revealable fields are omitted from operator source projections, but visible raster/QR pixels can disclose their contents. Reviews and acceptance are RAM-only; restart loses them. Do not write payloads into URLs, logs or template defaults.

7.6. Tests

`cargo test --workspace` includes real-HTTP simulator tests for both new types, asset retrieval, front/back separation, stale review handling and manual flip gating. `app/tests/ui.cjs` exercises the generic template editor and side controls using synthetic data. Safari/Firefox rendering still needs separate checks.

8. K6 Raster Protocol

This reference describes packet layouts generated or consumed by the current Rust `k6` driver and the device behavior observed while testing it. It is not a description of a vendor application's internals. The driver supports raster work; vectors and firmware updates are not part of this interface.

8.1. Link and Coordinates

- Serial device: normally `/dev/ttyUSB0`; baud: 115200.
- Pixel pitch: 0.05 mm (20 pixels/mm).

- Current normal K6 working reach: 80 x 75 mm. The Y value is an operator-selected bound, not an exact measurement of the mechanical stop.
- The firmware-coordinate envelope is 1600 x 1600 pixels. Normal jobs are constrained by the smaller working profile.
- Raster rows are packed MSB-first; each row is padded to a whole byte. Black pixels are set bits.
- For a raster with top-left `(x, y)`, the header center is `center_x = x + width/2 + 67`, `center_y = y + height/2`.

8.2. Response Bytes

Bytes	Observed meaning
`09`	ACK for commands/chunks that the driver expects to acknowledge.
`08`	Error response; observed when a DATA payload is entirely zero.
`FF FF FF FE`	Device activity/heartbeat frame, including after a job header.
`FF FF 00 xx`	Progress frame; `xx` is the reported percentage byte.

The Rust driver accepts `FF FF 00 64` (100) as reported completion. If no byte arrives for 90 seconds, it reports assumed completion and may proceed to the next band. Silence is not device confirmation or proof of physical rest.

8.3. Commands Used by the Driver

Name	Packet	Driver behavior
STOP	`16 00 04 00`	Writes the stop frame. During an active burn this is also sent through the stop handle while cancellation is latched. It does not by itself prove motion stopped.
VERSION	`FF 00 04 00`	After STOP and input drain, reads three version bytes. Observed firmware: `04 01 06`.
CONNECT	`0A 00 04 00`	Sent twice; each command expects `09`.
HOME	`17 00 04 00`	Expects `09`.
BOUNDS	`20 00 0B` + width, height, center X and center Y as four big-endian `u16` values	Laser-off frame request; expects `09`.
FRAMING	`21 00 04 00`	Expects `09`; ends the frame mode in the observed workflow.
JOB_HEADER	38-byte `0x23` packet described below	The driver waits for an activity/status frame; this packet has no ACK contract in the driver.

Name	Packet	Driver behavior
DATA	`0x22` packet described below	Up to 1900 payload bytes. Each chunk expects `09`; timeout/error can retry that chunk up to three attempts with backoff.
INIT	`24 00 0B 00 00 00 00 00 00 00 00`	Sent twice after DATA with approximately 200 ms, then 500 ms spacing before status monitoring.

A normal burn band sends FRAMING, JOB_HEADER, a size-dependent delay, two CONNECT commands and the DATA chunks, followed by two INIT commands. The driver then monitors status frames. A multi-band raster repeats the band sequence for each nonblank band.

A normal machine stop also sends STOP, then FRAMING and HOME. An in-burn stop may only reach the immediate stop handle and cancellation path. The operator must confirm physical rest before changing placement or continuing.

8.4. JOB_HEADER (0x23)

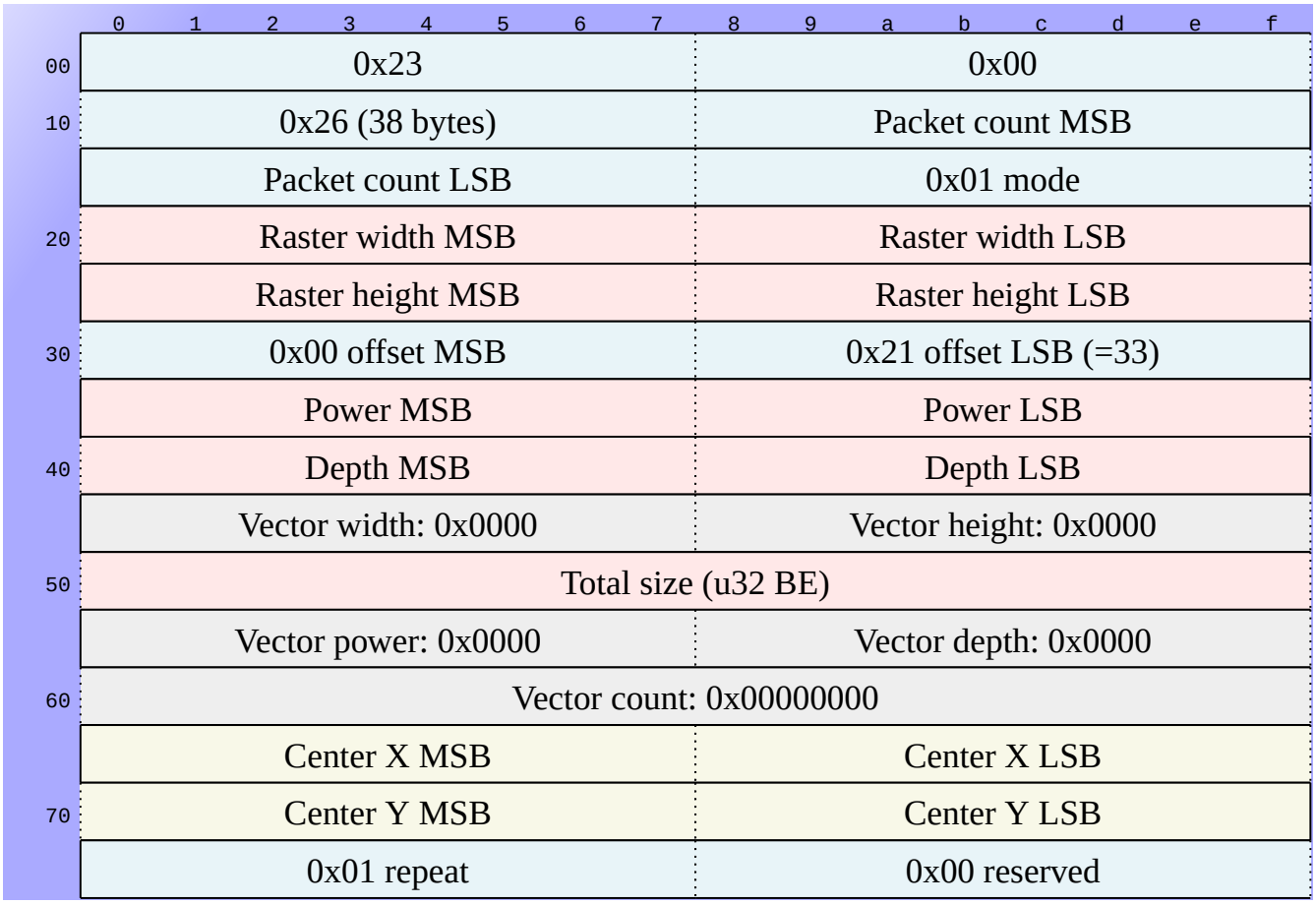


Figure 9. K6 raster JOB_HEADER layout; vector-reserved bytes are zero in the Rust driver

All multibyte fields are big-endian. The current raster-only builder sets:

Bytes	Field	Current value
0-2	Opcode, reserved, length	`23 00 26` (38 bytes)

Bytes	Field	Current value
3-4	Packet count	`(total_size / 4094) + 1`, integer division
5	Mode	`01`
6-7	Raster width	Pixels, `u16`
8-9	Raster height	Pixels, `u16`
10-11	Raster data offset	`00 21` (33)
12-13	Power	`u16`, machine setting
14-15	Depth	`u16`, machine setting
16-19	Vector width/height slots	Zero
20-23	Total size	Raster bytes + 33, `u32`
24-31	Vector power/depth/count slots	Zero
32-33	Center X	`u16`, pixels
34-35	Center Y	`u16`, pixels
36	Repeat	`01`
37	Reserved	`00`

The 33-byte data offset, raster-only values and repeat value describe the current Rust packet builder. Do not infer vector support from the reserved fields.

8.5. DATA (0x22)

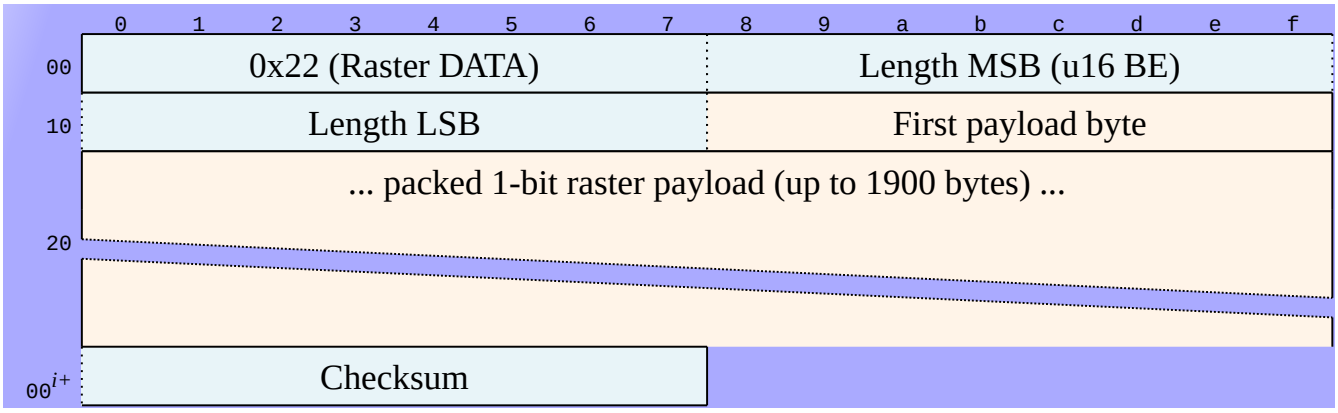


Figure 10. K6 raster DATA layout

- Bytes 0-2: opcode `22` and total packet length as a big-endian `u16`.
- Bytes 3 through the byte before the end: up to 1900 bytes of packed raster payload.
- Final byte: two's-complement checksum. The sum of the complete packet is zero modulo 256.

The driver drops blank edge rows and splits at long blank runs so that no transmitted 1900-byte chunk is all zero. This is a workaround for the observed `08` response to an all-zero DATA payload. It can result in several independent bands for one reviewed raster; progress percentages are band-local.

8.6. Framing, Stop and Status Diagrams

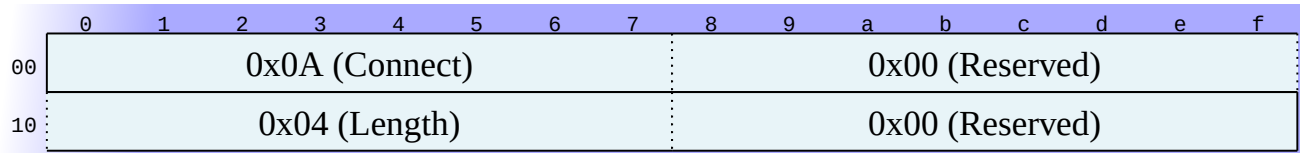


Figure 11. Connect

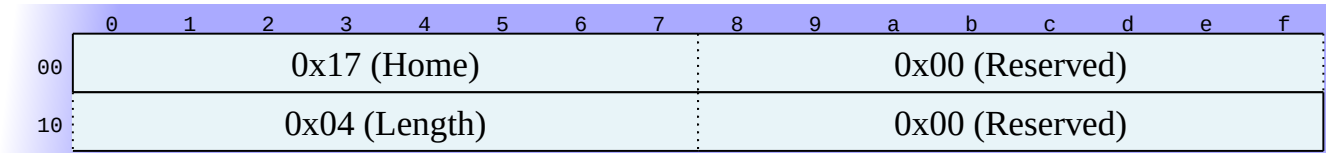


Figure 12. Home

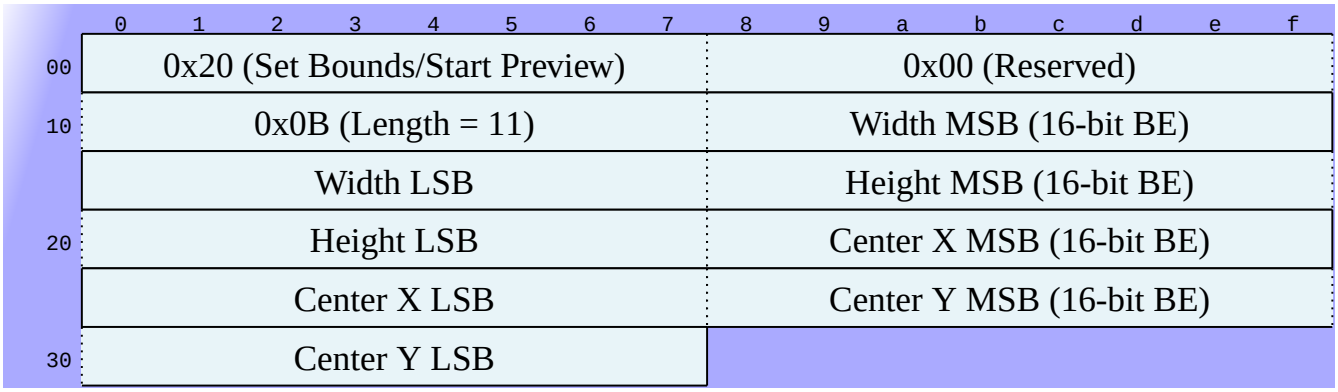


Figure 13. BOUNDS

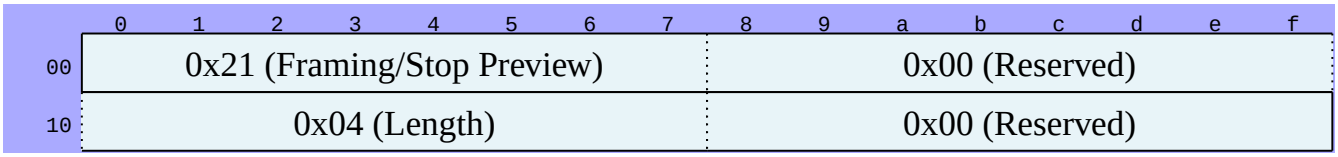


Figure 14. FRAMING

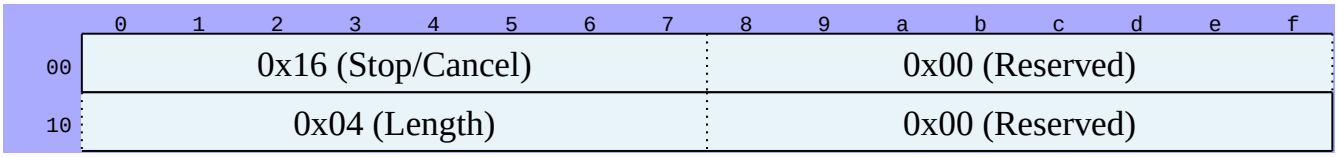


Figure 15. STOP

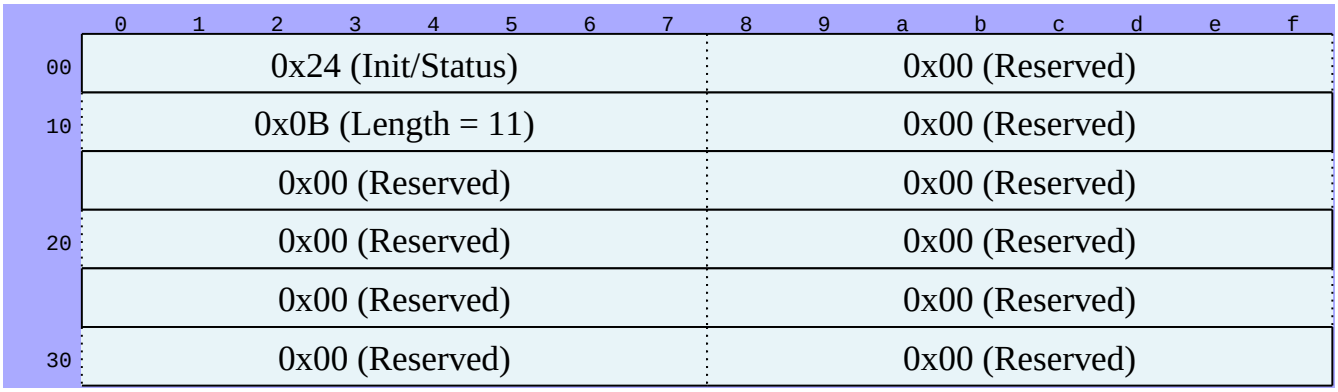


Figure 16. INIT

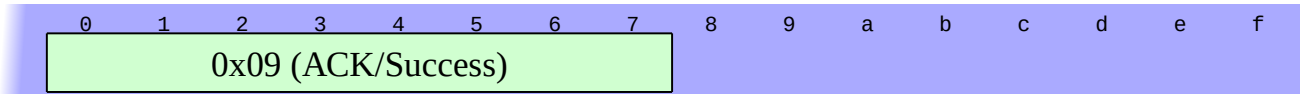


Figure 17. ACK and error

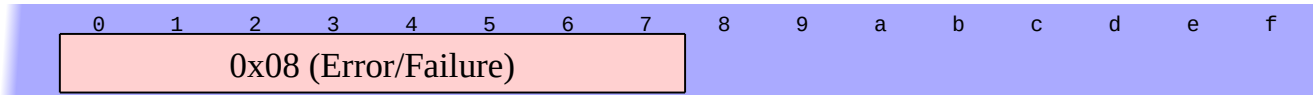


Figure 18. Error response byte

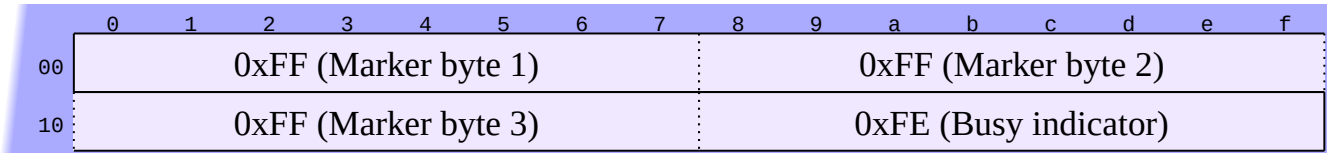


Figure 19. Activity and progress frames

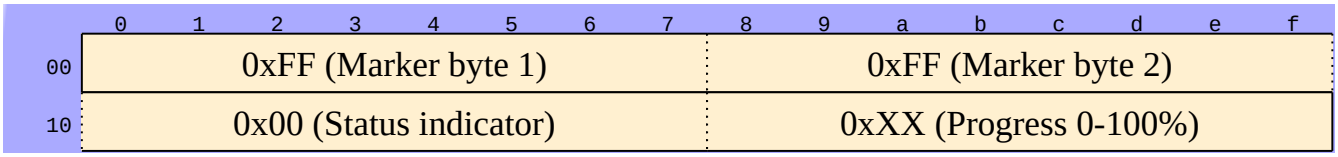


Figure 20. Band progress status frame

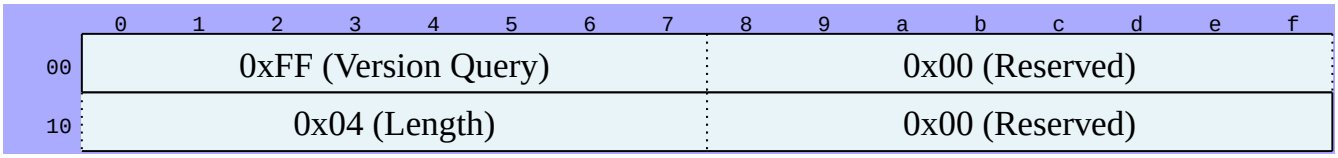


Figure 21. Version request and response

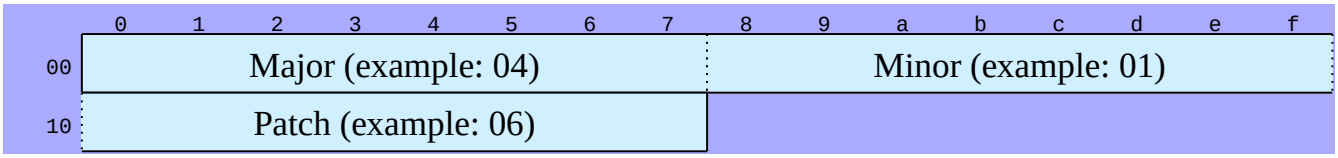


Figure 22. VERSION response

8.7. Evidence Limits

Packet fixtures and implementation are in `docker/docker-wainlux-rs/k6/src/protocol.rs`; sequencing and response handling are in `k6/src/device.rs`. The K6 packet SVGs are generated from the adjacent EDN sources with `scripts/generate_diagrams.sh`.

The Rust implementation and tests establish what this driver sends and parses. Hardware Observations distinguishes selected working limits and measured outcomes from assumptions. No command in this reference is a certified safety control.

9. Hardware Observations

Sean Donnellan triggered and verified the physical tests summarized here. This page keeps measurements that affect the current Rust service. Values are labeled by evidence type; historical runs are not current performance guarantees.

9.1. Current K6 Profile

Value	Evidence
Working reach: 80 x 75 mm; 20 px/mm	Current profile in `k6::info`. Sean selected 75 mm as the normal Y limit from the 9b comparison; it is not a measured hard stop.
Bamboo NFC card: 85.45 x 53.72 mm	Physically measured; configured as `bamboo-ntag215-light`.
Card material settings: power 1000, depth 10	Current configured profile. Safe depth and NFC survival remain material-specific acceptance checks.
Backing board dimensions	Not measured. `backing-first` dimensions are a planning placeholder, not a physical measurement.

9.2. Executed Rust Ladder Tests

Sean Donnellan triggered the real-device runs and supplied the observations/verdicts below. Software status and elapsed time are reported separately from the physical observation.

K6 hardware test ladder: historical rungs 0 through 12

Climb from rung 0 at the bottom to rung 12 at the top. Python and Rust results are kept distinct.

#	TEST	EXPECTED CHECK	RECORDED EVIDENCE
12	QR	Scan the engraved backing QR.	18.9 mm backing QR scanned first try, 2026-11-03.
11	Registration	Compare repeated marks for placement drift.	No physical verdict recovered.
10	Continuous	Compare continuous perimeter with the gapped run.	Historical Python: pass; no separate Rust verdict recorded.
9	Corners	Check corner squares and bottom-edge placement.	Deviation at 77.45 mm; 9b selected 75 mm normal Y.
8	Blank rows	Preserve position across empty raster rows.	Rust: pass with band split; unbanded upload got 0x08.
7	Limit squares	Compare seven square edges near the Y limit.	Square 4 intact; 5-7 clipped; 77.45 mm superseded.
6	Limit stairs	Repeat at higher settings; tune slope, then fine-tune.	Originally OK; superseded by 9b; discrepancy unknown.
5	First burn	Small filled square marks on backing.	Python and Rust: pass; 5 mm, 500/10, 102 s.
4	Frame	Laser-off bounds frame exits cleanly.	Python and Rust: pass after explicit exit sequence.
3	Pointer	Toggle aiming dot without head motion.	Deviation: no visible toggle; Rust rung was skipped.
2	Home	Head moves home and stops.	Python and Rust: pass; homing observed OK.
1	Talk	K6 replies to version/connect queries.	Python and Rust: pass; firmware reply 04 01 06.
0	Historical #0	Original rung label and goal were not recovered.	Python: pass; no Rust rung-0 result recorded.

Observed OK for the stated run · Performed deviation · Provisional boundary / software assumption · No verified result

Historical passes: Python 0, 1, 2, 4, 5, 10; Rust 1, 2, 4, 5, 6, 7, 8. Rust rung 3 was skipped; physical deviations were noted at 3 and 9.

A software pass is not a physical verdict. See Hardware Observations for run conditions, 9b measurements and Sean's verdicts.

Figure 23. Historical test ladder, climbed from rung 0 at the bottom to rung 12 at the top

Historical rung / test	Conditions / software report	Sean's observation / verdict
Rung 1: Talk	Rust ladder; firmware reply `04 01 06`.	Talk observed OK.
Rung 2: Home	Rust ladder.	Homing observed OK.
Rung 3: Pointer	Python pointer test had no visible toggle; Rust ladder marked this K6 rung skipped.	Deviation for the attempted behavior; not a Rust physical pass.
Rung 4: Frame	Laser-off bounds frame; ended with STOP, FRAMING, HOME.	Frame observed OK after the explicit exit sequence; an earlier frame attempt deviated.
Rung 5: First square	Filled 5 mm square on backing; power 500, depth 10; 102 s; status stalled at 99%.	Square intact; OK.
Rung 6: Limit stairs	74-80 mm Y, 80 mm width. 300/5 was unreadable; 500/10 showed a faint kink near 45 mm. Repeated at higher settings; slope was adjusted and fine-tuned until the pattern yielded an interpreted result. Exact later settings and endpoint are not in the current record.	Originally marked OK after tuning. Its result did not match 9b; the cause is unknown.
Rung 7: Limit squares	Attempt `1793779971-limit_squares`; power 1000, depth 10; requested bottoms 77.15-77.75 mm.	Square 4 intact, squares 5-7 clipped. Initial 77.45 mm decision was superseded by the 9b comparison.
Rung 8: Blank rows	First unbanded upload received `08` for an all-zero DATA chunk. Band-split workaround: 205 s at 500/10.	Two squares correctly placed, with no marks between; OK with workaround.
Rung 9: Corners	80 x 77.45 mm, power 1000, depth 10; two bands; 5849 s (97.5 min); status stalled at 99%.	Top squares good. Bottom squares compressed against/below the bottom line; Deviation on bottom placement. Cause was not established.
Rung 10: Continuous perimeter	Historical Python ladder result was pass.	No separate Rust physical verdict recovered; the 9b continuous-strip comparison is recorded below.
Rung 11: Registration	Rust step exists in the current ladder.	No physical verdict recovered.
Rung 12: QR	Backing QR, 377 px (18.9 mm), power 500 / depth 10.	Scanned on the first attempt.

Rung 6 was repeated at higher values, first adjusting the slope and then fine-tuning it, and was marked OK at the time. The later 9b comparison did not agree with its apparent limit: both 77.45

mm strips deviated while the 75 mm gap and continuous strips were OK. No cause for the discrepancy is established. The 9b result is the accepted basis for the 75 mm normal Y profile; it does not measure the hard stop.

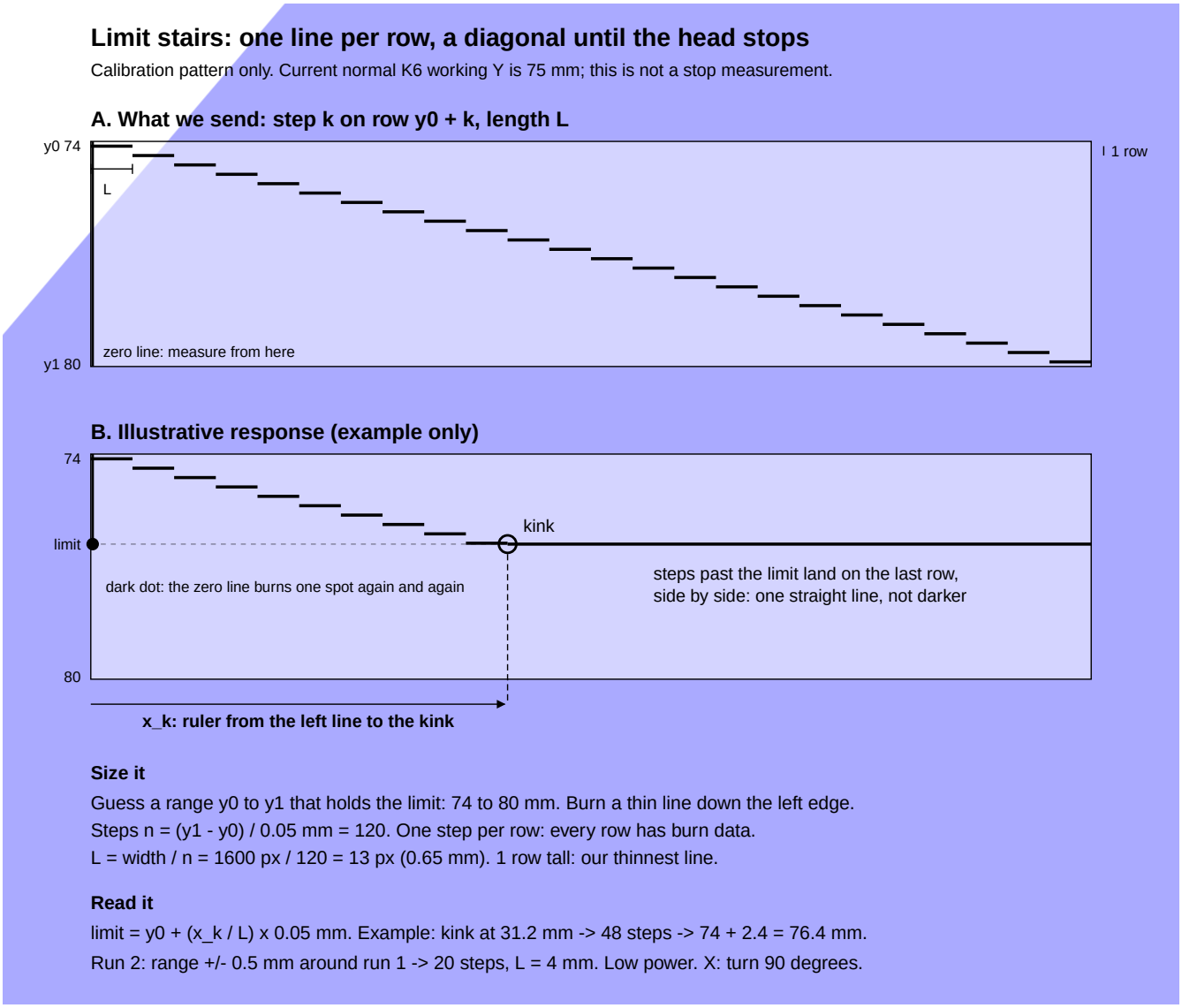


Figure 24. Limit stairs: requested calibration pattern, not a measured result

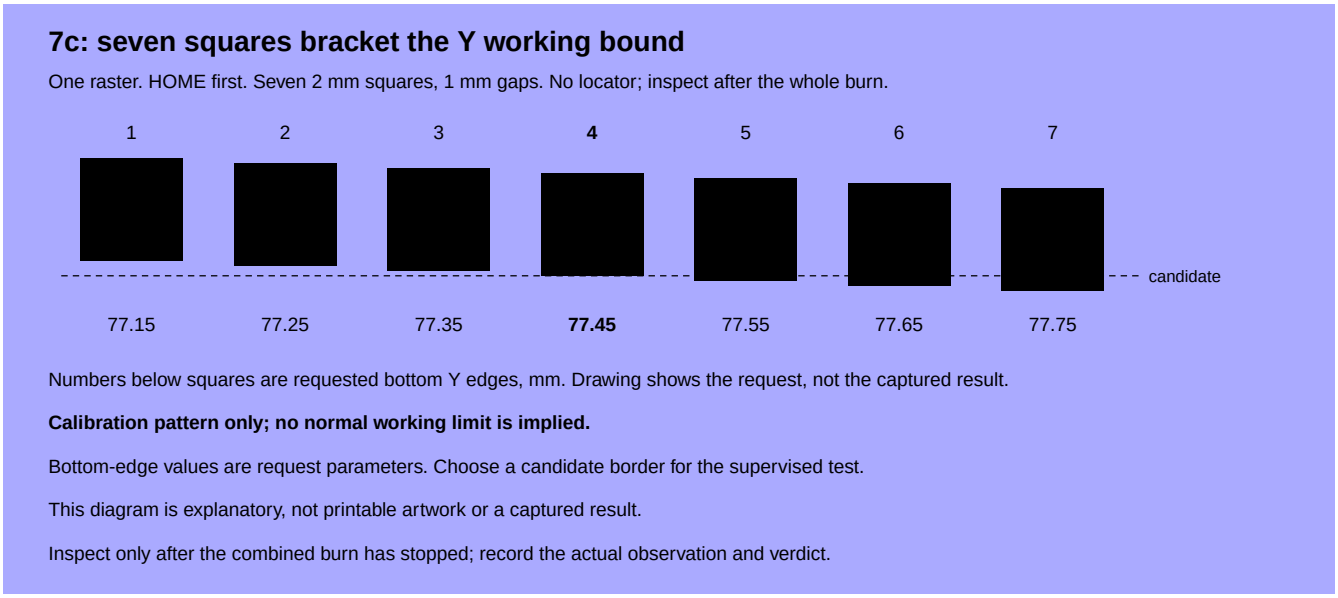


Figure 25. Seven-square calibration pattern: 77.45 mm was provisional and later superseded

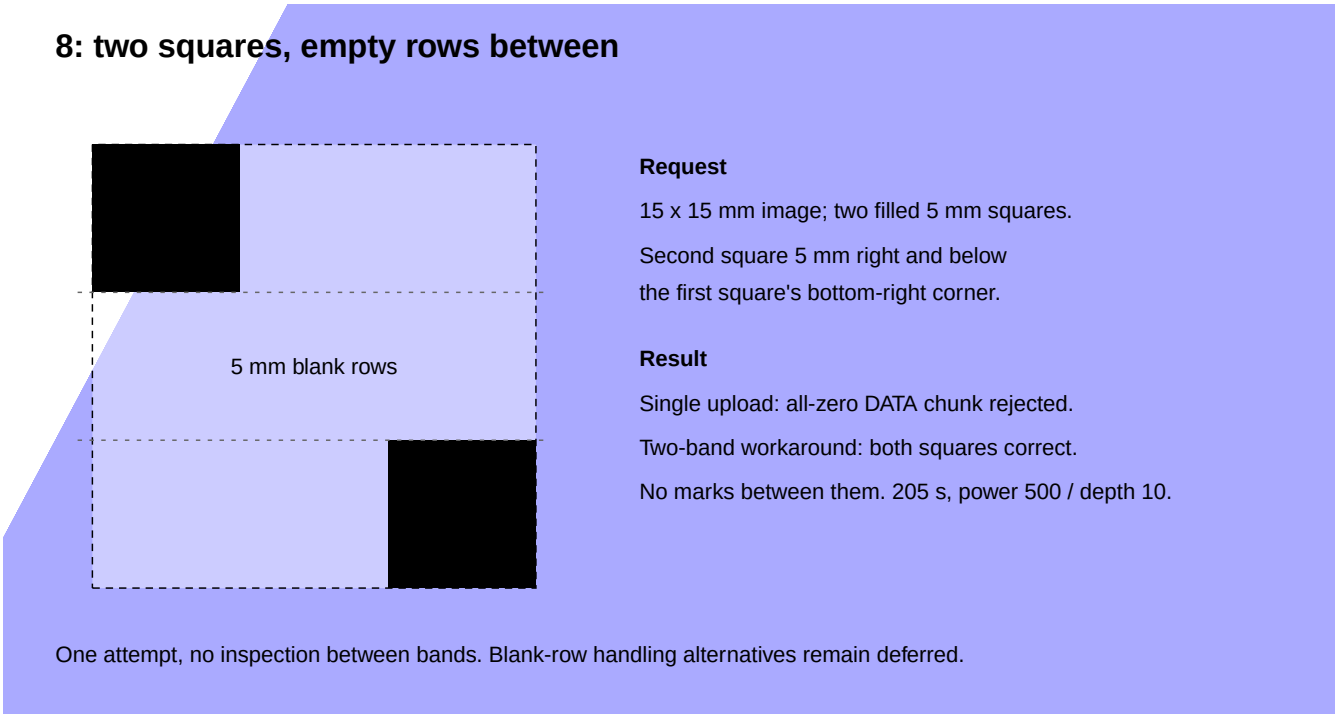


Figure 26. Blank-row test pattern

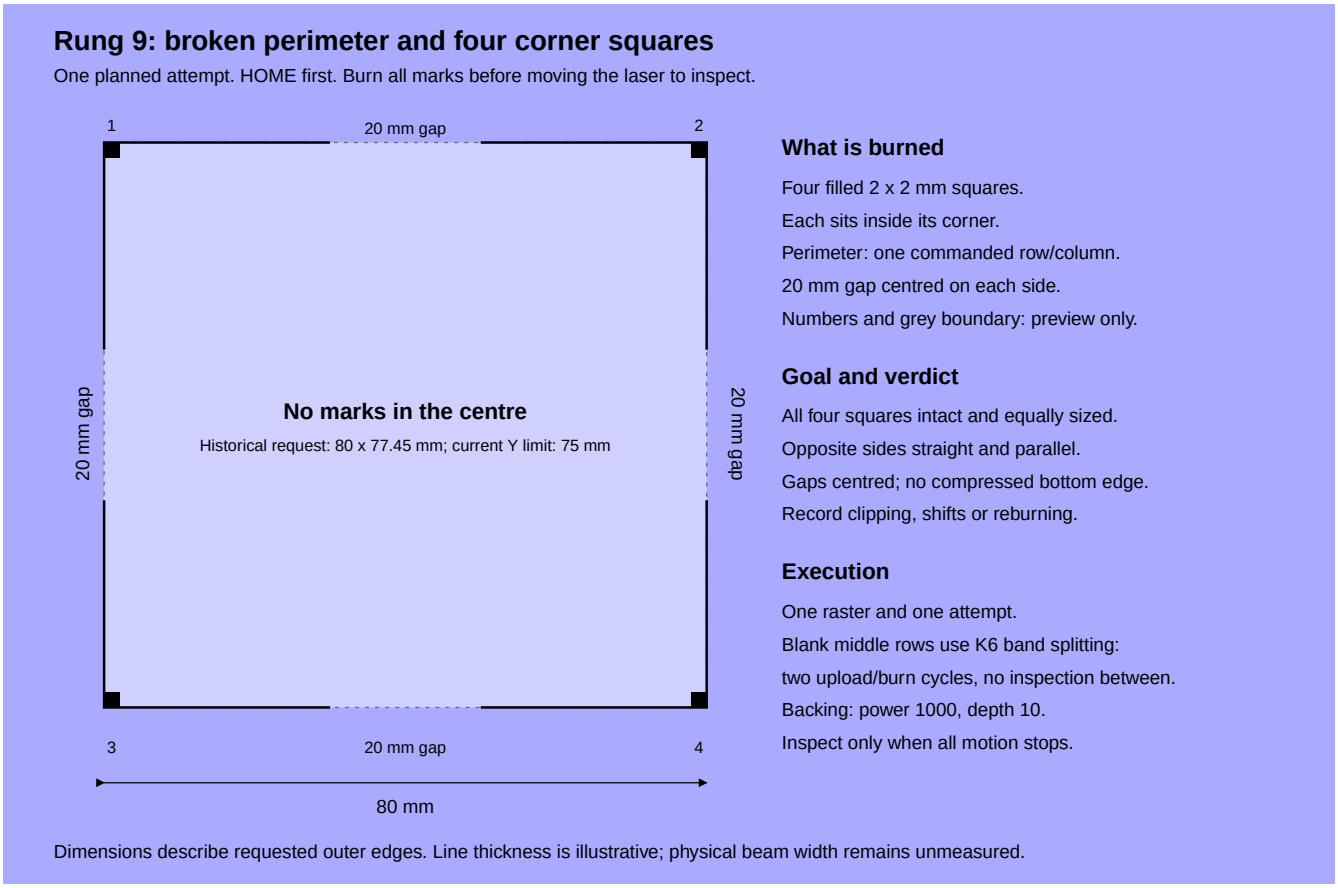


Figure 27. Corner test pattern at the former 77.45 mm height

9.3. 9b: Height and Gap Comparison

A/B/C were one joint session on the stock base. D was a later follow-up on the same setup. Each strip was a separate job with HOME before it; the backing was not repositioned. All used power 1000 and depth 10. Software status stalled near 99% in all four runs.

Strip	Pattern	X	Height	Duration	Sean's verdict / observation
A	20 mm gap	10 mm	77.45 mm	223 s	Deviation: bottom marker compressed.
B	Continuous line	20 mm	77.45 mm	242 s	Deviation: same bottom compression as A.
C	20 mm gap	30 mm	75 mm	219 s	OK, approximately 75 mm.
D	Continuous line, later follow-up	40 mm	75 mm	237 s	OK; matched C.

Sean's joint inspection found the gap and continuous pair matched at each height. The 77.45 mm pair deviated at the bottom; the 75 mm pair was OK. Decision: 75 mm normal Y working bound. This did not measure the mechanical hard stop or prove every multi-band placement.

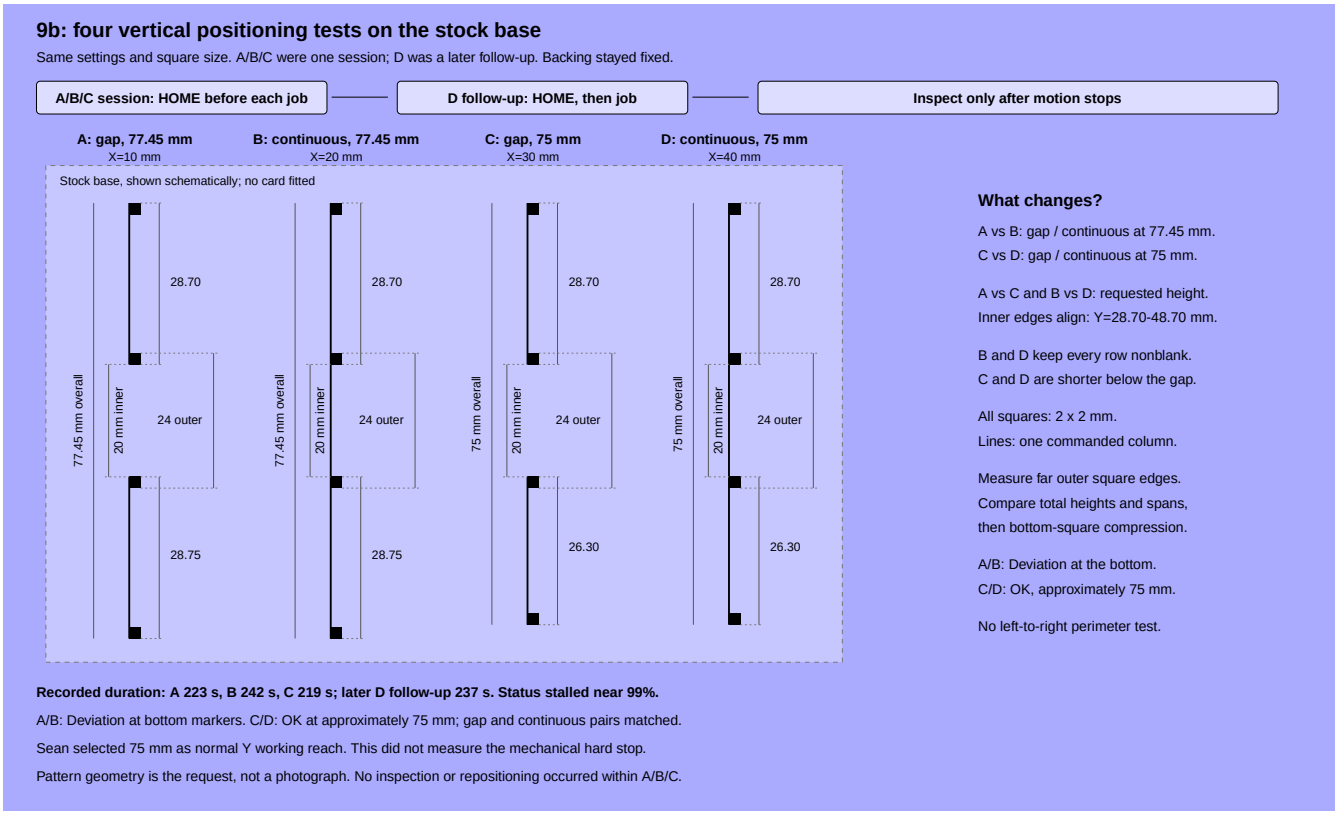


Figure 28. 9b four-strip comparison geometry; D was a later follow-up

9.4. Card Result

Sean reported that the first card run had a good guide, readable QR and working NFC. Printed text was too small. The subsequent larger bold text defaults have not been physically accepted in a physical run; verify them on a new supervised card before treating them as measured results.



Figure 29. Generic open-right card guide used on bare backing

9.5. Historical Cork and PLA Close-ups

These recovered images show material behavior from earlier experiments. They are microscope/camera close-ups of cork and PLA, not the missing close-ups from the 9b corner/limit run.

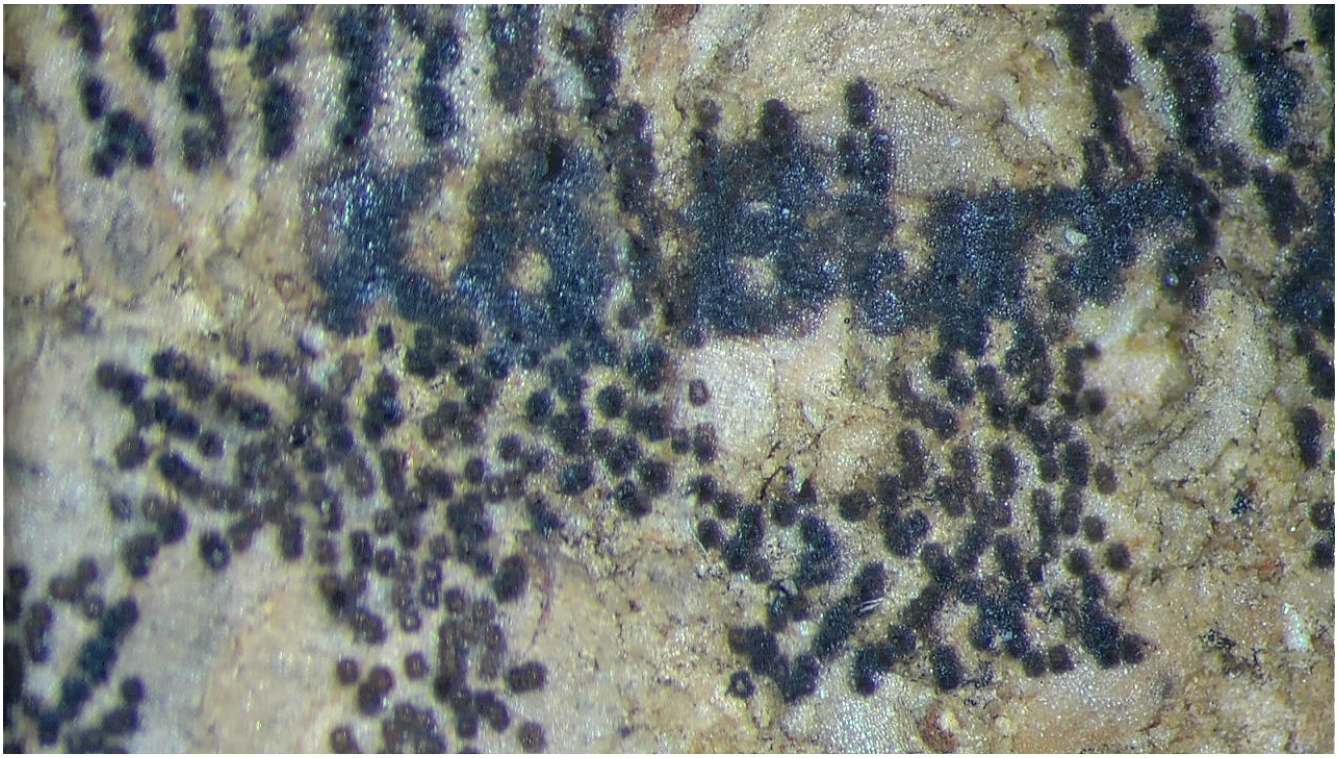


Figure 30. Cork test-pattern surface

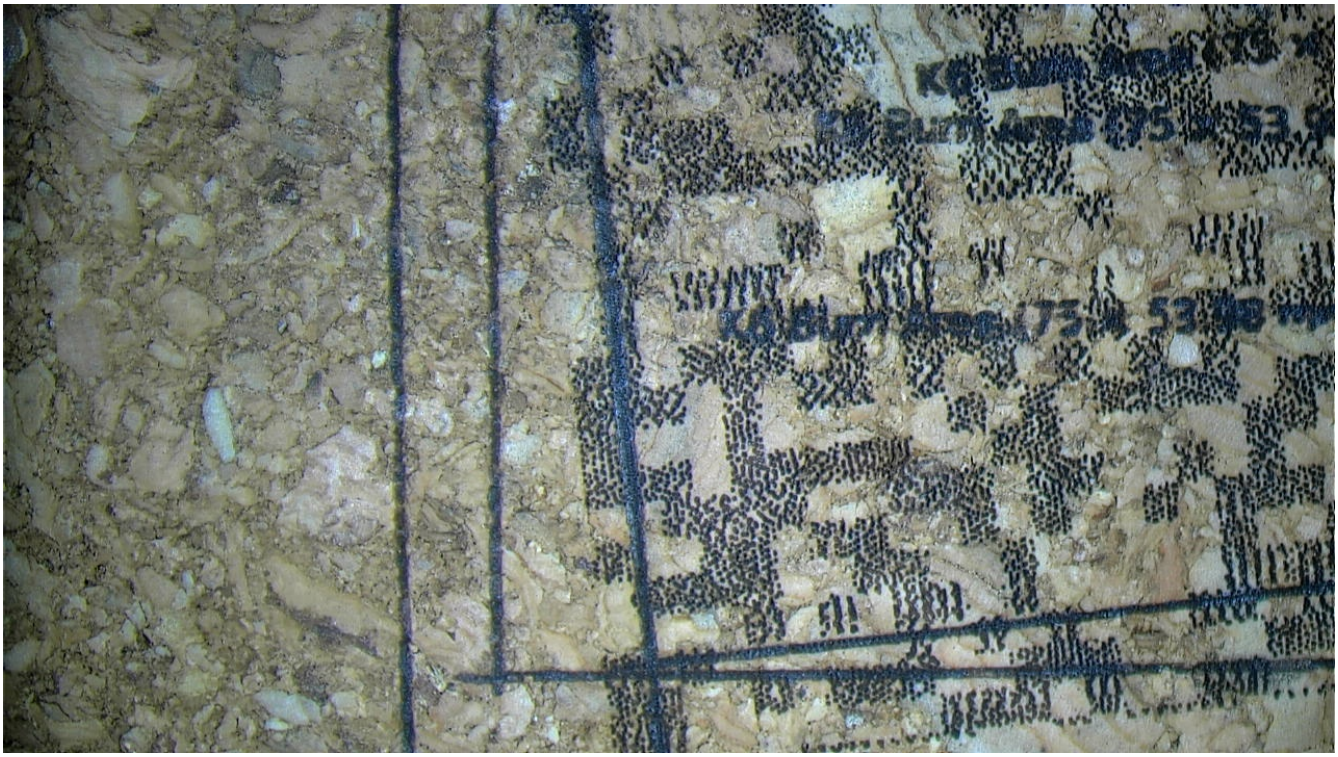


Figure 31. Cork pattern at wider view

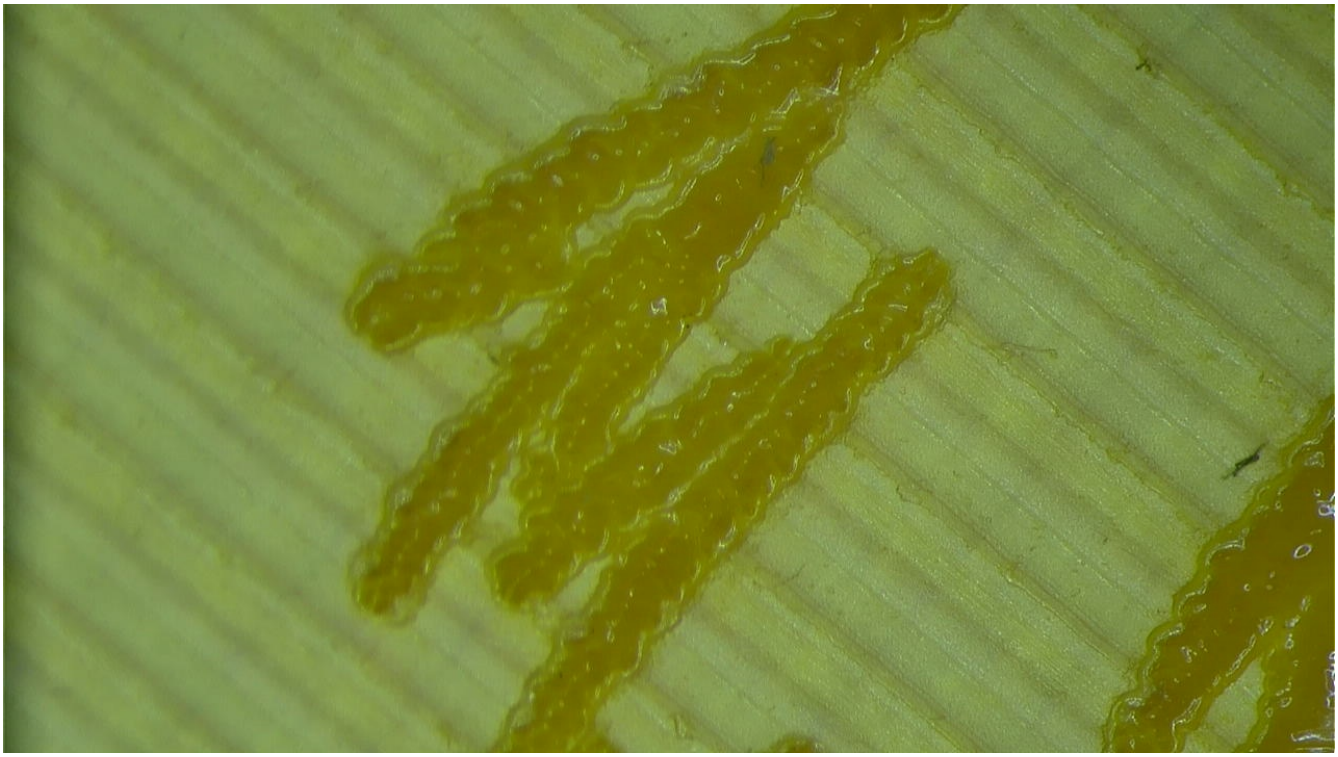


Figure 32. Pulsed line detail on PLA



Figure 33. Tiger pattern on PLA

9.6. Historical QR Run

A QR on the backing board was 377 px (18.9 mm), burned at power 500 / depth 10, and scanned on

the first attempt. This was a backing-board run, not the current card template's 36 mm QR.

For the historical transfer-time capture, see the provenance and measurements in `doing-debugging.adoc`.

10. Burn Timing

10.1. Current Report

The Rust service exposes completed band timings in `GET /api/state` as `band_timings` and includes them in the run outcome. Each record describes one raster band. For a multi-band run, sum the records for a run total.

Field	Meaning
`transfer_ms`	Time spent sending raster data, including any retries.
`setup_ms`	Raster preparation and protocol setup outside transfer and completion wait.
`wait_ms`	Time waiting for K6 completion status after setup.
`elapsed_ms`	Total elapsed time for the band.
`phase`	`reported_complete` means the device reported completion; `assumed_complete` means the service observed 90 seconds of silence.
`percent`	Device-reported progress for this band; it is not the percentage for the full side or card.

The timing values are milliseconds. Small differences between summed component times and elapsed time can result from measurement boundaries and millisecond rounding. Assumed completion is not device confirmation.

10.2. Historical Measurements

The following measurements are from the retired Python service and are not Rust performance claims.

- A 1600x1600 raster used a 320,000-byte payload in 169 chunks. Transfer took 35.8 seconds, about 8.7 KB/s, with approximately 208-210 ms per chunk.
- In that run, the first observed completion status arrived about 4.9 seconds after data transfer. The first observed percentage was 37%; earlier progress was not captured.
- In a separate completion-detection run, the estimated duration was 336 seconds while the observed burn lasted 889 seconds. The old monitor stopped early; this observation informed later completion handling.

These are dated observations from specific runs. Use `band_timings` to see how a current Rust run divides time among setup, transfer and completion wait.

Appendix A: Architecture Decisions

Only decisions that remain in force for the Rust service are listed here. Sean Donnellan approved these project decisions; superseded implementation history was removed from the working docs, with provenance left to Git history.

Clean-room methodology is summarized in Clean-Room Statement. * ADR-002: K6 serial transport * ADR-006: Clean-room protocol work * ADR-007: AsciiDoc * ADR-010: Bytefield diagrams * ADR-013: Simulator testing * ADR-019: HTTP review boundary * ADR-021: Rust service * ADR-022: ARM packaging * ADR-025: Card placement * ADR-026: Network MCP hub

11. ADR-002: K6 Serial Transport

11.1. Status

Accepted; current.

11.2. Decision

The Rust `k6` crate owns all K6 protocol framing and communicates through the Linux serial device, normally `/dev/ttyUSB0` at 115200 baud. The CP210x bridge is accessed through the kernel serial driver; the application does not speak USB directly.

The `laser::Machine` contract keeps serial details out of composition and HTTP handling. Host tests use the simulator and do not require a serial device.

11.3. Consequences

- Only the K6 driver opens the serial port.
- A second process must not own `/dev/ttyUSB0` at the same time.
- Packet evidence and observed byte layouts are in the K6 protocol reference.

12. ADR-006: Clean-Room Protocol Work

12.1. Status

Accepted; ongoing constraint.

12.2. Decision

Describe observed bytes and device behavior, not vendor source structure or implementation. The current Rust service is independently implemented from observable protocol behavior. Decompiled material was used only as a verification aid; it is not included or translated into the service.

12.3. Consequences

- Keep packet captures, hardware observations and uncertainty distinct from implementation claims.
- Do not copy vendor code, names, comments or design structure.
- The methodology and contributor boundary are summarized in Clean-Room Statement. This record makes no legal conclusion.

13. ADR-007: AsciiDoc

13.1. Status

Accepted; current.

13.2. Decision

Use AsciiDoc for repository documentation, with PlantUML for process diagrams and Bytefield-SVG for packet layouts. Keep sources beside the documents that use them.

13.3. Consequences

- Validate documentation with `./scripts/test_docs.sh` from the repository root.
- Keep generated diagrams reproducible from checked-in source where a generator exists.

14. ADR-010: Bytefield Protocol Diagrams

14.1. Status

Accepted; used by the K6 protocol reference.

14.2. Decision

Keep packet layouts as EDN specifications and generated SVGs. `scripts/generate_diagrams.sh` renders the specifications in `documentation/images/` with the `docker/docker-bytefield` container.

14.3. Consequences

- EDN is the editable source; SVG is the rendered documentation asset.
- These diagrams document observed packet layouts. They are not runtime configuration or a claim that every documented opcode is exposed by the Rust service.
- The Pi does not need Node.js or Bytefield-SVG.

15. ADR-013: Simulator Testing

15.1. Status

Accepted; current.

15.2. Decision

Use the Rust simulator for API and workflow tests that must not access the K6. `MACHINE=sim` removes the serial-device dependency; tests exercise the HTTP contract against that driver.

This is simulation, not a real-device dry-run. Real mode can move and burn. A preview or simulator verdict is not evidence of physical completion.

15.3. Consequences

- Keep hardware-independent tests deterministic and explicit about their simulator driver.
- Do not describe the retired Python dry-run toggle or its operation modes as Rust features.
- Physical acceptance remains an operator observation recorded separately from software outcome.

16. ADR-019: HTTP Review Boundary

16.1. Status

Accepted; current, with an explicit trust limit.

16.2. Decision

The Pi service is the single owner of the K6 serial port. A producer may submit a job for validation and preview through HTTP; submission does not execute it. The operator reviews the raster, then separately plans and runs each stage and records a verdict.

The API is for a trusted LAN. Producer and operator roles are not authenticated, and there is no per-client ownership lease. This boundary is workflow separation, not access control.

16.3. Consequences

- Browser and external clients use the same HTTP contract; neither accesses serial directly.
- Do not expose the service to an untrusted network.
- Reviews are held in process memory; run, stop and verdict events are appended to `DATA_DIR/runs.jsonl` without submitted job payloads.
- Restart loses the active review; the JSONL log is not a job database or restart-recovery

mechanism.

17. ADR-021: Rust Service

17.1. Status

Accepted; current.

17.2. Decision

The Pi service is a Rust workspace with four responsibilities:

- `app`: HTTP API, review/run/verdict workflow and embedded UI.
- `compose`: pure content, placement and raster generation.
- `k6`: K6 serial protocol driver.
- `laser`: machine, raster, settings, progress and stop contracts, plus simulator.

The service handles raster jobs. Each user stage requires a separate plan and run request; a later card side requires the prior stage verdict and explicit placement confirmation.

17.3. State and lifetime

Active review and attempt state are in process memory and disappear on restart. The service appends redacted run, stop, mode and verdict events to `DATA_DIR/runs.jsonl`. It does not persist submitted QR/text payloads or raster jobs. Public assets and TOML definitions are files, not user job state.

17.4. Consequences

- Protocol and hardware behavior stay in the K6 driver; composition is testable without a laser.
- There is no job queue, per-client session owner or restart recovery. These are current limits, not promised features.
- The canonical API contract is included at [api-contract] and served at `GET /api`.

18. ADR-022: ARM Packaging

18.1. Status

Accepted; current.

18.2. Decision

Build the Pi ARMv6 container away from the Pi. The Rust cross-build targets `arm-unknown-linux-musleabihf`; the runtime image copies in the binary and configuration. GitHub Actions also builds the ARMv6 image from `docker/docker-wainlux-rs/Dockerfile`.

The Pi Compose file in `docker/docker-wainlux-rs/compose.yaml` runs the loaded `localhost/laser-app:arm` image, mounts `/dev/ttyUSB0` and persistent `data/`, and serves port 8081. Compose does not compile the service on the Pi.

18.3. Consequences

- Pi Zero W is a runtime target, not a Rust build host.
- Keep image architecture, target triple and serial-device mapping aligned with the actual Pi deployment.
- Build and deployment commands belong with the service README and workflow; the retired Bash deployment scripts are not part of this decision.

19. ADR-025: Card Placement

19.1. Status

Accepted; current.

19.2. Decision

Placement uses millimetres and the machine's declared reach. The current K6 profile is 80 x 75 mm. For the measured 85.45 x 53.72 mm bamboo card, the default left placement uses a 1 mm margin; its centered reachable etch area is 72.55 x 53.72 mm.

The optional guide is generated only on sides that fit wholly inside reach. For the default card placement it is an open-right U, with a 0.5 mm line 0.2 mm outside the piece. Items are positioned in the card coordinate frame and translated to bed coordinates. Any content outside the etch area is refused, not clipped.

19.3. Consequences

- Materials and per-machine settings are declared in `docker/docker-wainlux-rs/materials.toml`.
- Use the 80 x 75 mm profile for normal jobs. The 75 mm Y value is an operator-selected working bound; it is not an exact stop measurement.
- The configured backing dimensions are a placeholder until the physical board is measured.

20. ADR-026: Network MCP Hub

20.1. Status

Accepted by Sean Donnellan. Current placement is a separate preview-only service on the Pi Zero host; relocation remains possible.

20.2. Decision

Run the MCP server as a separate Streamable HTTP process/container on the Pi Zero host for now. It is not part of the Rust app process and is not a stdio child of the IDE. It is an HTTP client of the Rust service; it never opens `/dev/ttyUSB0`. The same service can later move to an independent MCP hub without changing its API role.

The first tool slice is machine/material/template discovery and non-mutating job preview. It does not create or replace a retained review and has no plan, run, stop or verdict tools. The operator UI remains the only supported path for inspecting exact raster pixels and starting work.

20.3. Consequences

- The MCP container uses the Pi-local Rust API by default. Its Streamable HTTP endpoint is reachable to trusted Wi-Fi clients on port 8082.
- Keep it on the private network and use firewall/TLS/access controls where available; the service API itself has no authentication.
- The current MCP and laser APIs have no authentication. Do not expose either publicly.
- Future mutating MCP tools require an explicit ADR and authorization design; do not add burn controls by implication.

Appendix B: Requirements

Only requirements backed by current Rust behavior or an explicit safety boundary remain. Superseded Flask requirements and unimplemented proposals were removed; numbering gaps are intentional.

B.1. Functional

Historical physical evidence is in Hardware Observations. * FR-001: Reviewed raster execution * FR-002: WiFi QR composition * FR-003: Progress and timing * FR-007: Stop and cancellation * FR-009: Test ladder * FR-011: Material placement * FR-013: Simulator testing * FR-015: Operator test verdicts * FR-016: Card review workflow

B.2. Non-Functional

- NFR-002: Hardware-free test isolation

- NFR-005: Redacted run history

21. FR-001: Reviewed Raster Execution

21.1. Status

Implemented; see the service API contract and HTTP tests.

21.2. Requirement

Compose supported card content into bounded 1-bit raster stages. A user must be able to inspect the exact pixels and placement before execution. Planning and running are separate operations.

21.3. Acceptance

- Review responses contain exact per-stage PNG/SVG pixels, dimensions and physical placement.
- Settings and geometry are validated; content outside the declared etch area is rejected, not clipped.
- A plan names a reviewed stage; only a separate run request accesses the selected machine.
- Software outcome and operator verdict are recorded separately.

21.4. Scope

Content kinds are QR, WiFi QR, text, prepared public assets and checkboxes. Arbitrary image uploads and vector burns are not part of this contract. See the Rust service README and `app/tests/api.rs`.

22. FR-002: WiFi QR Composition

22.1. Status

Implemented in the `wifi-login` use case; physical acceptance is material and phone dependent.

22.2. Requirement

Compose a WiFi payload as a QR and preview the resulting raster before a stage can run.

22.3. Acceptance

- Accept SSID, password, security type and hidden-network flag through declared fields or the typed API.
- Encode credentials in the QR; never bind the password to printed text.

- Keep QR size and position within the selected material's etch area; reject overflow.
- Return preview pixels and dimensions before planning or running.

22.4. Evidence

The included API contract defines the template behavior. Physical results and their limits are in Hardware Observations.

23. FR-003: Progress and Timing

23.1. Status

Implemented; Rust API and UI tests cover the structured report.

23.2. Requirement

Report progress and elapsed work without implying that one band percentage is whole-job progress or that software completion proves physical rest.

23.3. Acceptance

- `/api/state.progress` identifies phase, band, source rows and band-local percentage.
- Completed bands appear in `band_timings` and the run outcome with `transfer_ms`, `setup_ms`, `wait_ms` and `elapsed_ms`.
- Reported device completion and assumed completion after 90 seconds of silence remain distinct.
- The UI labels waiting and assumed completion explicitly.

All durations are milliseconds. Historical Python timings are not performance targets for Rust. See Burn Timing.

24. FR-007: Stop and Cancellation

24.1. Status

Software cancellation implemented; physical stop behavior is not certified.

24.2. Requirement

A stop request must cancel queued work and request a device stop when a device handle exists. It must not automatically retry, start another band or authorize a later stage.

24.3. Acceptance

- Cancellation is checked during connection, upload, wait and between bands.
- The API reports whether a stop request was sent and whether work was cancelled.
- Errors, STOP and uncertain completion do not trigger automatic job, stage or band retries. The driver may retry an individual DATA chunk according to the protocol retry policy.
- An operator confirms motion has stopped before changing placement or proceeding.

A successful HTTP response is not proof of physical rest. Use the machine's physical emergency controls and supervision; this API is not a certified emergency-stop system.

25. FR-009: Test Ladder

25.1. Status

Implemented for the K6 and simulator; individual steps have machine capability gates.

25.2. Requirement

Expose named machine tests with a stated goal and preview where available. Tests must use the same Rust machine contract as card jobs.

25.3. Acceptance

- `/api/steps` lists the steps supported by the selected machine.
- Supported steps include communication, home, frame, square, limits, blank rows, corner/registration comparisons and QR tests.
- A plan records the expected goal before execution; the API does not label a physical test OK on the operator's behalf.
- Rasters used by tests obey the machine's declared bounds unless the step explicitly tests the calibration envelope.

Run results, selected measurements, the 0–12 ladder and pattern illustrations are in Hardware Observations.

26. FR-011: Material Placement

26.1. Status

Implemented in the Rust composition layer.

26.2. Requirement

Represent the piece, backing, guide and content geometry in millimetres. Refuse any generated burn rectangle outside the K6's declared reach or the piece's etch area.

26.3. Acceptance

- Materials define dimensions, role and machine-specific settings.
- Placement supports the configured centered and left layouts; guide segments are emitted only where they fit.
- Content positions are relative to the piece and translated to bed coordinates.
- Out-of-area content is rejected without clipping or silent resizing.

The current K6 normal profile is 80 x 75 mm. Card dimensions and placement assumptions are in ADR-025 and the included API contract.

27. FR-013: Hardware-Independent Testing

27.1. Status

Implemented through the Rust simulator and workspace tests.

27.2. Requirement

Run API, composition and workflow tests without opening a physical K6 serial port.

27.3. Acceptance

- Tests select the simulator driver and bind only to a local test HTTP port.
- Tests cover review, plan, run, verdict, stage gating and redacted logs with synthetic data.
- Simulator results cannot be treated as evidence of burn quality, device stop or physical completion.

This is simulator testing, not a real-device dry-run mode.

28. FR-015: Operator Test Verdicts

28.1. Status

Implemented for planned test steps and card stages.

28.2. Requirement

Record the goal before a run, the software outcome after it, and an independent operator verdict. Preserve the original goal when an observation differs.

28.3. Acceptance

- A plan fixes the test goal before the machine runs.
- `POST /api/run` executes only the planned attempt.
- `POST /api/verdict` records `ok`, `deviation` or `skipped`; deviation and skipped require a comment.
- `ok` requires an outcome. A software outcome alone is not a physical verdict.
- Later reviewed card stages require acceptance of earlier stages; a back stage also requires explicit placement confirmation.

Use the simulator for automated contract tests. Real-device checks require an attended operator and a recorded physical observation.

29. FR-016: Reviewed Card Jobs

29.1. Status

Implemented for WiFi, vCard and GitHub recovery templates; integration and physical acceptance remain use-case dependent.

29.2. Requirement

Accept a typed card job, compose and return separate reviewed stages, and require explicit operator action for each physical stage.

29.3. Acceptance

- Supported templates are supplied by TOML definitions; API clients may submit typed content or edit declared fields.
- Review returns side-specific raster pixels, dimensions, layout and display policy without opening the serial device.
- Front and back are separate stages. A back stage requires an accepted preceding stage and explicit flip/placement confirmation.
- Only separate plan and run requests execute a stage. Software outcome and operator verdict remain distinct.
- Private job/review data stays in process memory and is omitted from operational logs; restart loses the review.

29.4. Trust Limit

The producer submission and machine APIs are unauthenticated and intended for a trusted LAN. There is no per-client ownership, lease or producer/operator authorization. Masking is a UI convenience, not access control. Do not expose this service publicly.

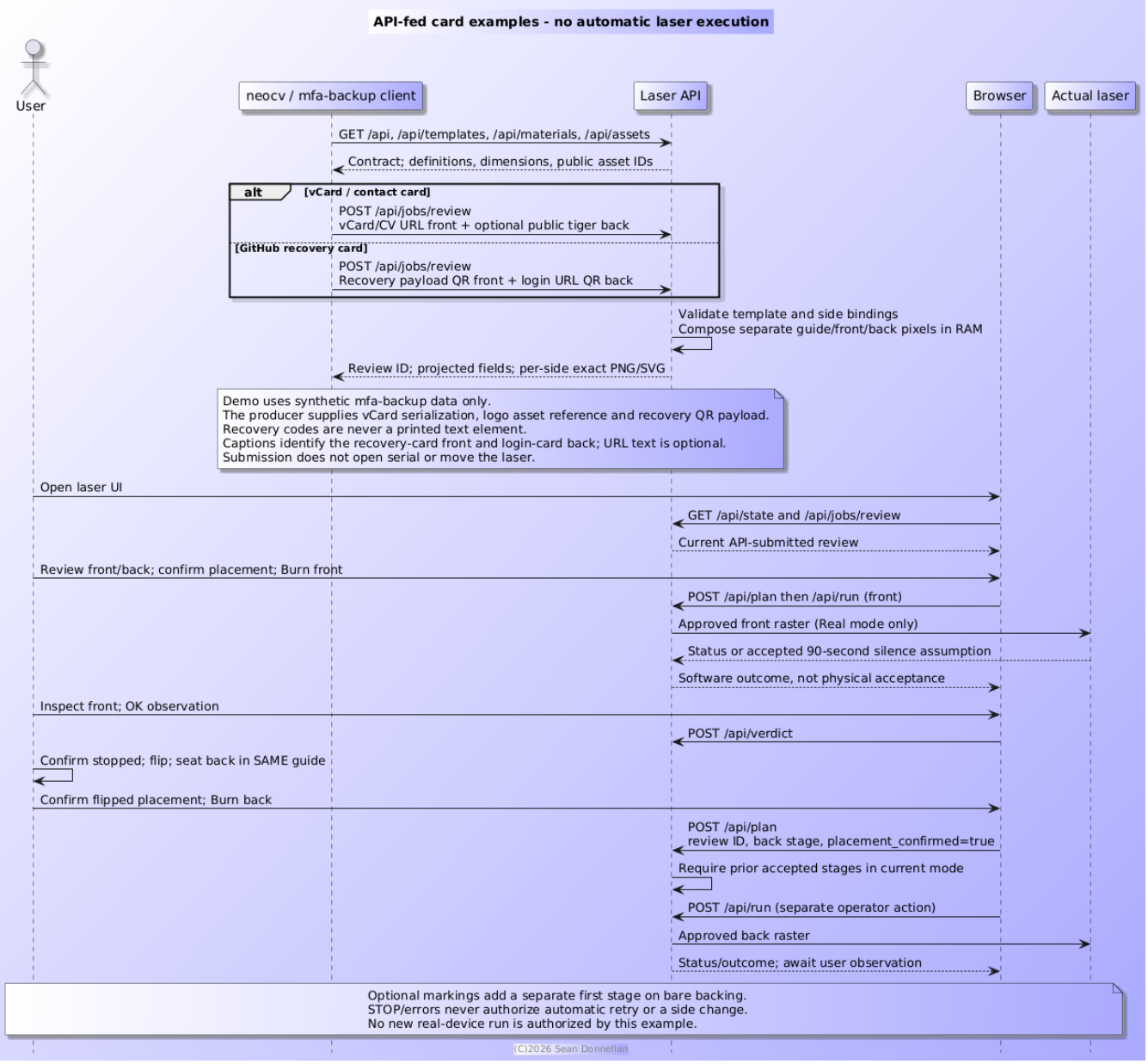


Figure 34. Current API submission, review, and operator-controlled run sequence

30. NFR-002: Hardware-Free Test Isolation

30.1. Status

Implemented for simulator-backed automated tests. No real-device dry-run exists.

30.2. Requirement

Automated software tests must be able to exercise the API and composition path without connecting to or moving a laser.

30.3. Acceptance

- Workspace and HTTP contract tests use the simulator and synthetic payloads.
- Simulator mode does not open `/dev/ttyUSB0` or send machine commands.
- A simulator result is never presented as physical acceptance.

A preview is not a real-device no-burn mode. Real mode can burn material and requires operator supervision.

31. NFR-005: Redacted Run History

31.1. Status

Implemented for run, stop, mode and verdict events.

31.2. Requirement

Persist operational outcomes in an append-only JSONL file without persisting submitted card payloads or raster jobs.

31.3. Acceptance

- Events are appended to `DATA_DIR/runs.jsonl` and are available through `GET /api/log`.
- Run records retain a redacted goal, software outcome and band timings; verdict records retain the operator's status and comment.
- Submitted job and parameter objects are removed before logging; tests verify synthetic SSIDs and passwords do not appear in the log.
- The log is not a job database: active review state is in RAM and is lost on restart.

The log currently has no rotation or retention policy.

Appendix C: Images

Current diagrams used by the documentation.

- `k6-prot-*.edn` → `.svg` with Bytefield-SVG (`scripts/generate_diagrams.sh`).
- `system-context.plantuml`, `architecture-rust.plantuml`, `sequence-stage-run.plantuml`, `card-api-clients.plantuml` and `sequence-ghidra-analysis.plantuml` show the current device/client context,

service ownership, stage execution, API clients and clean-room handoff. Their PNGs are generated with PlantUML.

- `card-guide.svg` is a generic open-right guide with no submitted content or credentials.
- `card-placement.svg` and `card-items.svg` are generic illustrations of the current K6/card geometry and template coordinates.
- `ui-simulator-preview.png` is a Rust operator UI screenshot with synthetic Wi-Fi values, simulator mode and no burn started. The simulator's 80 x 80 mm profile is not the measured K6 working reach.
- `test-ladder.svg`, `test-case-07a-limit-stairs.svg`, `test-case-07c-seven-squares.svg`, `test-case-08-blank-rows.svg` and `test-case-09-corners.svg` show Rust test coverage/evidence. The 9b comparison is shared from `docker/docker-wainlux-rs/app/assets/test-patterns/vertical-comparison.svg`.
- `capture_20260127_124112.jpg`, `capture_20260127_124223.jpg`, `capture_20260127_124337.jpg` and `capture.2426611769506763096.jpg` are recovered historical cork/PLA close-ups; they are not 9b ladder microscope captures.

Measured results are in Hardware Observations. Unverified design proposals and retired Python/UI diagrams have been removed.

Appendix D: Glossary

ACK

Acknowledgement byte (`0x09`) observed from the K6 after a successful command.

ARMv6

32-bit ARM architecture used by the Pi Zero W.

Assumed completion

Software outcome after 90 seconds without a received device byte. It is not device confirmation or an operator verdict.

Band

A contiguous group of raster rows transferred and burned as one unit. Progress percentages and timing records are band-local.

Calibration envelope

Coordinate range used by an explicit limit test. It may exceed normal job reach and does not establish the mechanical hard stop.

CP2102

USB-to-serial bridge family observed in the K6 connection; Linux exposes the serial device to the service.

Depth

K6 laser on-time setting per pixel. The supported setting range is 1-255; higher values increase

exposure.

Docker

Container runtime used to package the service. Compose maps the serial device into the container.

K6

Wainlux K6 laser engraver controlled over USB serial. The configured normal working profile is 80 x 75 mm; this is not a measured mechanical stop.

NFC

Near-field communication used by compatible contactless cards such as the measured NTAG215 card.

Operator verdict

Human `ok`, `deviation`, or `skipped` record for a completed or unperformed plan. It is distinct from the software outcome.

Plan

A record of the reviewed stage and expected goal. Planning does not start machine motion.

Pi Zero W

Raspberry Pi Zero W, an ARMv6, 512 MB Wi-Fi host for the service.

QR quiet zone

Blank margin around a QR symbol that helps scanners locate and decode it.

Raster

One-bit image represented as rows of pixels; K6 transfer data packs eight pixels per byte.

Review

Composed job with layout and exact per-stage rasters. Creating or editing a review does not start the laser.

Stage

One separately reviewed and explicitly started unit of a card workflow, such as a guide, front, or back.

Working reach

Configured normal job rectangle. For the current K6 profile it is 80 x 75 mm; the mechanical end stop has not been measured.

Appendix E: Copyright and License

E.1. Code License

All source code in this repository is licensed under the **MIT License**.

See MIT License for the full text.

E.2. Protocol Documentation License

All protocol documentation (files in `/documentation`) is dual-licensed:

- **Creative Commons Attribution 4.0 International (CC BY 4.0)**, or
- **Creative Commons Zero v1.0 Universal (CC0 1.0)** - Public Domain Dedication

See Protocol Documentation License for the full text.

E.3. Clean Room Statement

This project was developed through clean-room reverse-engineering methods. No vendor source code, proprietary binaries, or confidential materials were used or redistributed.

See Clean-Room Statement for methodology and exclusions.

E.4. Copyright Notice

Copyright (c) 2026 Sean Donnellan

Appendix F: MIT License

Full text of the repository's MIT code license

```
MIT License

Copyright (c) 2026 Sean Donnellan

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Appendix G: Protocol Documentation License

Full text of the protocol documentation license

```
Protocol Documentation License

All protocol documentation in the /documentation directory is licensed under
Creative Commons Attribution 4.0 International (CC BY 4.0).

You are free to:
- Share — copy and redistribute the material in any medium or format
- Adapt — remix, transform, and build upon the material for any purpose

Under the following terms:
- Attribution — You must give appropriate credit, provide a link to the
  license, and indicate if changes were made.

Full license text: https://creativecommons.org/licenses/by/4.0/legalcode

---

Alternatively, at your option, the protocol documentation may be used under
Creative Commons Zero v1.0 Universal (CC0 1.0) - Public Domain Dedication.

Full license text: https://creativecommons.org/publicdomain/zero/1.0/legalcode
```

Appendix H: Bytefield-SVG

The protocol packet diagrams are generated from the EDN sources in `documentation/images/` with the Bytefield-SVG container. See Bytefield-SVG Container and `scripts/generate_diagrams.sh`.

32. Bytefield-SVG Container

Docker container for generating protocol diagrams using bytefield-svg.

32.1. Build

Build the Bytefield-SVG container

```
cd docker-bytefield
host-spawn podman build -t bytefield-svg .
```

32.2. Usage

Generate diagram from JSON spec:

Render a JSON packet specification

```
host-spawn podman run --rm -v $(pwd):/diagrams:Z bytefield-svg input.json -o
output.svg
```

Generate from Clojure EDN spec:

Render an EDN packet specification

```
host-spawn podman run --rm -v $(pwd):/diagrams:Z bytefield-svg input.edn -o output.svg
```

32.3. Helper Script

Use `scripts/generate_diagrams.sh` which handles host-spawn automatically.

Or create alias:

Define a local Bytefield command alias

```
alias bytefield='host-spawn podman run --rm -v $(pwd):/diagrams:Z bytefield-svg'
bytefield input.json -o output.svg
```

32.4. Flatpak Users

Uses `host-spawn podman` to access host system's podman installation. No additional Flatpak extensions needed.

32.5. Examples

See the included Images appendix for K6 protocol packet specs.